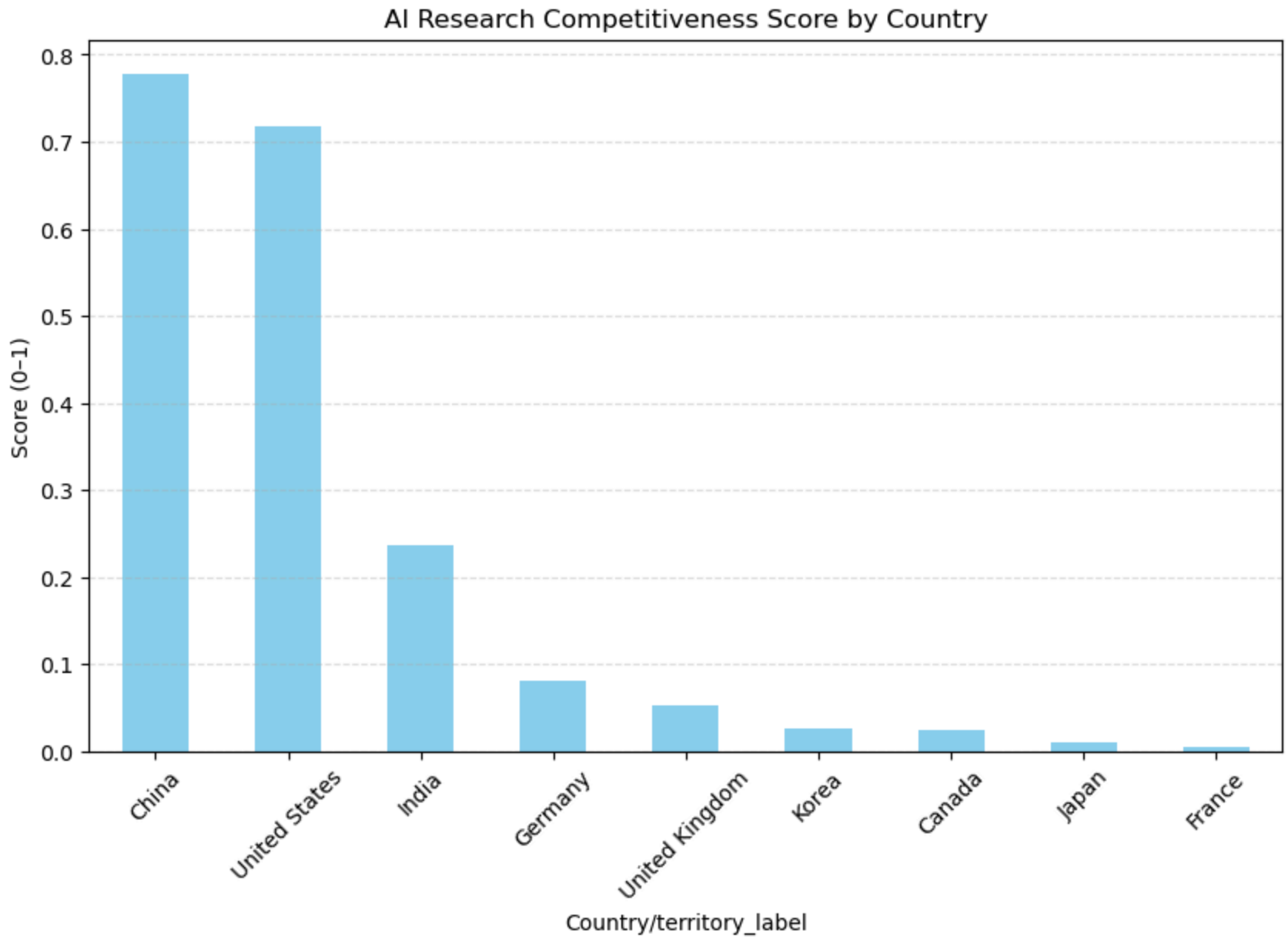
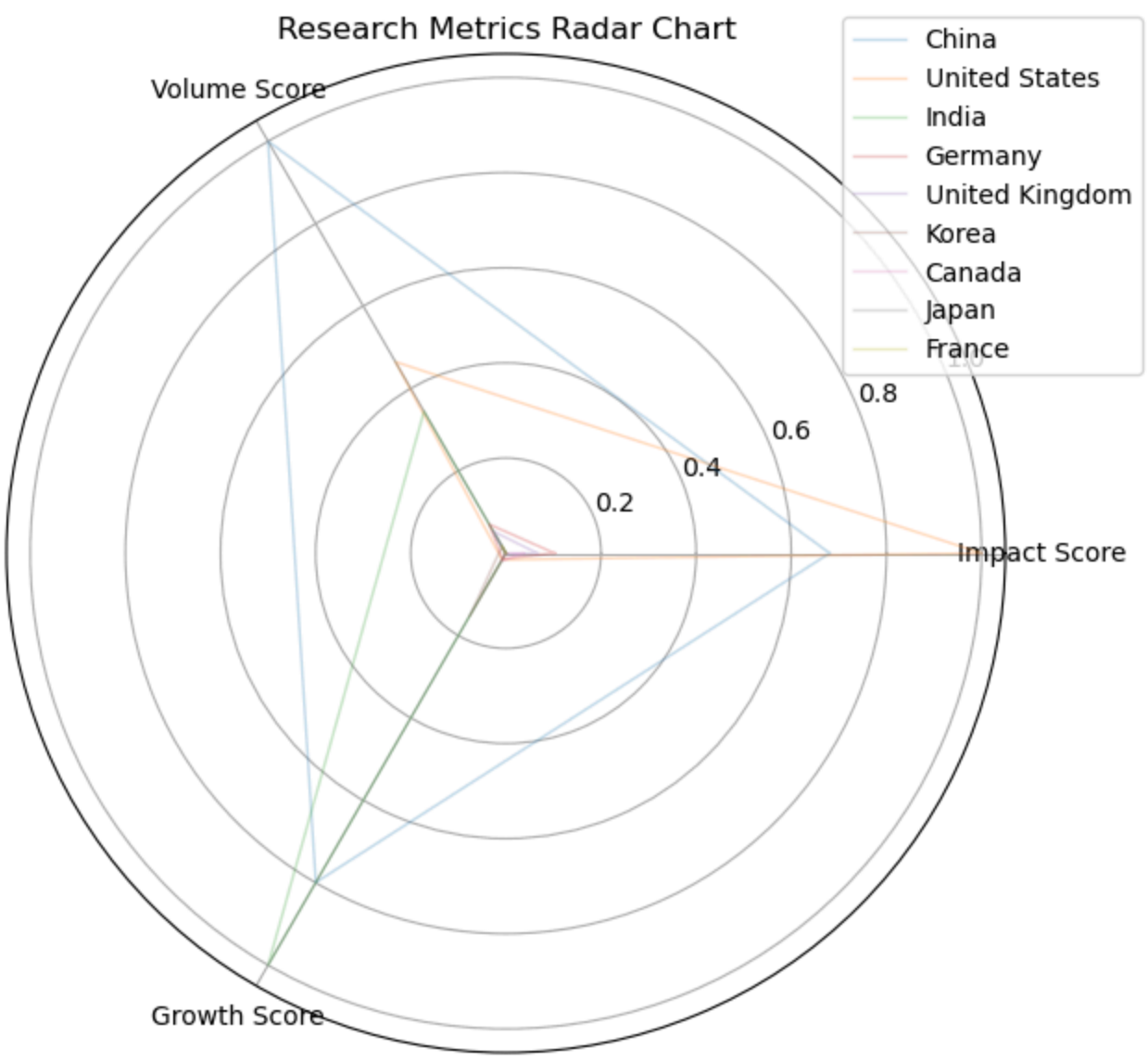
#1 Research

In [1]:
```python
# Version 1

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ========================================================
# 1. Load Data
# ========================================================
df_AI_Article = pd.read_csv("1AI Article.csv")
df_pub_raw = pd.read_csv("1Top countries in AI publications in time.csv")

print("=== Impact Dataset Head ===")
print(df_AI_Article.head())

print("\n=== Publication Dataset Head ===")
print(df_pub_raw.head())

# ========================================================
# 2. Standardize Country Names
# ========================================================
country_map = {
    "China (People's Republic of)": "China",
    "United States of America": "United States",
    "Republic of Korea": "South Korea",
    "Korea, Rep.": "South Korea",
    "Hong Kong": "China",
    "Taiwan": "China",
}

df_AI_Article["Geographic area"] = df_AI_Article["Geographic area"].replace(country_map)
df_pub_raw["Country/territory_label"] = df_pub_raw["Country/territory_label"].replace(country_map)

# ========================================================
# 3. Impact Score (High-impact research influence)
# ========================================================
df_imp = df_AI_Article.groupby("Geographic area")[
    "Number of highly cited publications in top 100"
].sum().reset_index()

df_imp["impact_share"] = (
    df_imp["Number of highly cited publications in top 100"]
    / df_imp["Number of highly cited publications in top 100"].sum()
)

impact_raw = df_imp.set_index("Geographic area")["impact_share"]
impact_score = (impact_raw - impact_raw.min()) / (impact_raw.max() - impact_raw.min())

# ========================================================
# 4. Publication Data Cleaning (Long → Wide)
# ========================================================
df_pub_filtered = df_pub_raw[
    (df_pub_raw["ai_concept"] == "All AI research") &
    (df_pub_raw["pub_type"] == "All publication types")
]

df_pub = df_pub_filtered.pivot_table(
    index="year",
    columns="Country/territory_label",
    values="publications",
    aggfunc="sum"
)

print("\n=== Pivoted Publication Table ===")
print(df_pub.head())

# ========================================================
# 5. Publication Volume Score (2021–2023)
# ========================================================
df_pub_recent = df_pub.loc[df_pub.index.isin([2021, 2022, 2023])]
pub_volume_raw = df_pub_recent.mean()

pub_volume_score = (pub_volume_raw - pub_volume_raw.min()) / (
    pub_volume_raw.max() - pub_volume_raw.min()
)

# ========================================================
# 6. Publication Growth Score (First → Last year)
# ========================================================
pub_growth_raw = (df_pub.iloc[-1] - df_pub.iloc[0]) / df_pub.iloc[0]

pub_growth_score = (pub_growth_raw - pub_growth_raw.min()) / (
    pub_growth_raw.max() - pub_growth_raw.min()
)

# ========================================================
# 7. Align Countries for Scoring
# ========================================================
countries = pub_volume_score.index
```

```python
impact_score = impact_score.reindex(countries, fill_value=0)
pub_growth_score = pub_growth_score.reindex(countries, fill_value=0)

# ========================================================
# 8. Compute Final Research Score
# ========================================================
research_score = (
    0.6 * impact_score +
    0.25 * pub_volume_score +
    0.15 * pub_growth_score
)

research_df = pd.DataFrame({
    "Impact Score": impact_score,
    "Volume Score": pub_volume_score,
    "Growth Score": pub_growth_score,
    "Research Score": research_score
}).sort_values("Research Score", ascending=False)

print("\n===== Final Research Score Table =====")
print(research_df)

# ========================================================
# 9. Visualization — Bar Chart
# ========================================================
plt.figure(figsize=(10, 6))
research_df["Research Score"].plot(kind="bar", color="skyblue")
plt.title("AI Research Competitiveness Score by Country")
plt.ylabel("Score (0–1)")
plt.xticks(rotation=45)
plt.grid(axis="y", linestyle="--", alpha=0.4)
plt.show()

# ========================================================
# 10. Visualization — Radar Chart
# ========================================================
from math import pi

metrics = ["Impact Score", "Volume Score", "Growth Score"]
N = len(metrics)

plt.figure(figsize=(7, 7))
angles = [n / float(N) * 2 * pi for n in range(N)]
angles += angles[:1]

for country in research_df.index:
    values = research_df.loc[country, metrics].tolist()
    values += values[:1]
    plt.polar(angles, values, linewidth=1, alpha=0.25, label=country)

plt.xticks(angles[:-1], metrics)
plt.title("Research Metrics Radar Chart")
plt.legend(bbox_to_anchor=(1.15, 1.05))
plt.show()
```

```
=== Impact Dataset Head ===
   Number of highly cited publications in top 100 Geographic area  Label
0                                              50   United States   2023
1                                              34           China   2023
2                                               7         Germany   2023
3                                               7       Hong Kong   2023
4                                               6          Canada   2023

=== Publication Dataset Head ===
        ai_concept Country/territory       Country/territory_label  \
0  All AI research               CHN  China (People's Republic of)
1  All AI research               CHN  China (People's Republic of)
2  All AI research               CHN  China (People's Republic of)
3  All AI research               CHN  China (People's Republic of)
4  All AI research               CHN  China (People's Republic of)

  impact_level               pub_type  publications  year
0    All works  All publication types    638.583334  2000
1    All works  All publication types    831.100001  2001
2    All works  All publication types   3143.697621  2002
3    All works  All publication types   2822.383338  2003
4    All works  All publication types   4152.416669  2004

=== Pivoted Publication Table ===
Country/territory_label       Canada        China       France      Germany  \
year
2000                     434.798486   638.583334   633.779366   842.194662
2001                     424.350000   831.100001   511.955557   873.214287
2002                    1854.115084  3143.697621  1846.916887  2422.616814
2003                     999.511113  2822.383338  1102.881820  1259.405557
2004                     813.142859  4152.416669  1087.210175  1289.352311

Country/territory_label       India        Japan        Korea  United Kingdom  \
year
2000                    154.179798  1162.583334   239.833334     1036.065045
2001                    144.166667  1109.333334   233.133334     1057.722224
2002                    559.782938  5140.350940  1019.116668     2721.166346
2003                    388.666667  2986.093420   597.350001     1826.264685
2004                    471.875001  2502.774173   634.883334     1520.409744

Country/territory_label  United States
year
2000                       3628.998705
2001                       3389.037307
2002                      14443.905785
2003                       8505.192256
2004                       5870.007980

===== Final Research Score Table =====
                         Impact Score  Volume Score  Growth Score  \
Country/territory_label
China                        0.680723      1.000000      0.799173
United States                1.000000      0.464509      0.015975
India                        0.000000      0.344188      1.000000
Germany                      0.102410      0.069978      0.014253
United Kingdom               0.066265      0.053056      0.002379
Korea                        0.000000      0.016360      0.151776
Canada                       0.036145      0.000000      0.019863
Japan                        0.000000      0.040829      0.000000
France                       0.000000      0.017309      0.000944

                         Research Score
Country/territory_label
China                          0.778310
United States                  0.718523
India                          0.236047
Germany                        0.081078
United Kingdom                 0.053380
Korea                          0.026856
Canada                         0.024666
Japan                          0.010207
France                         0.004469
```





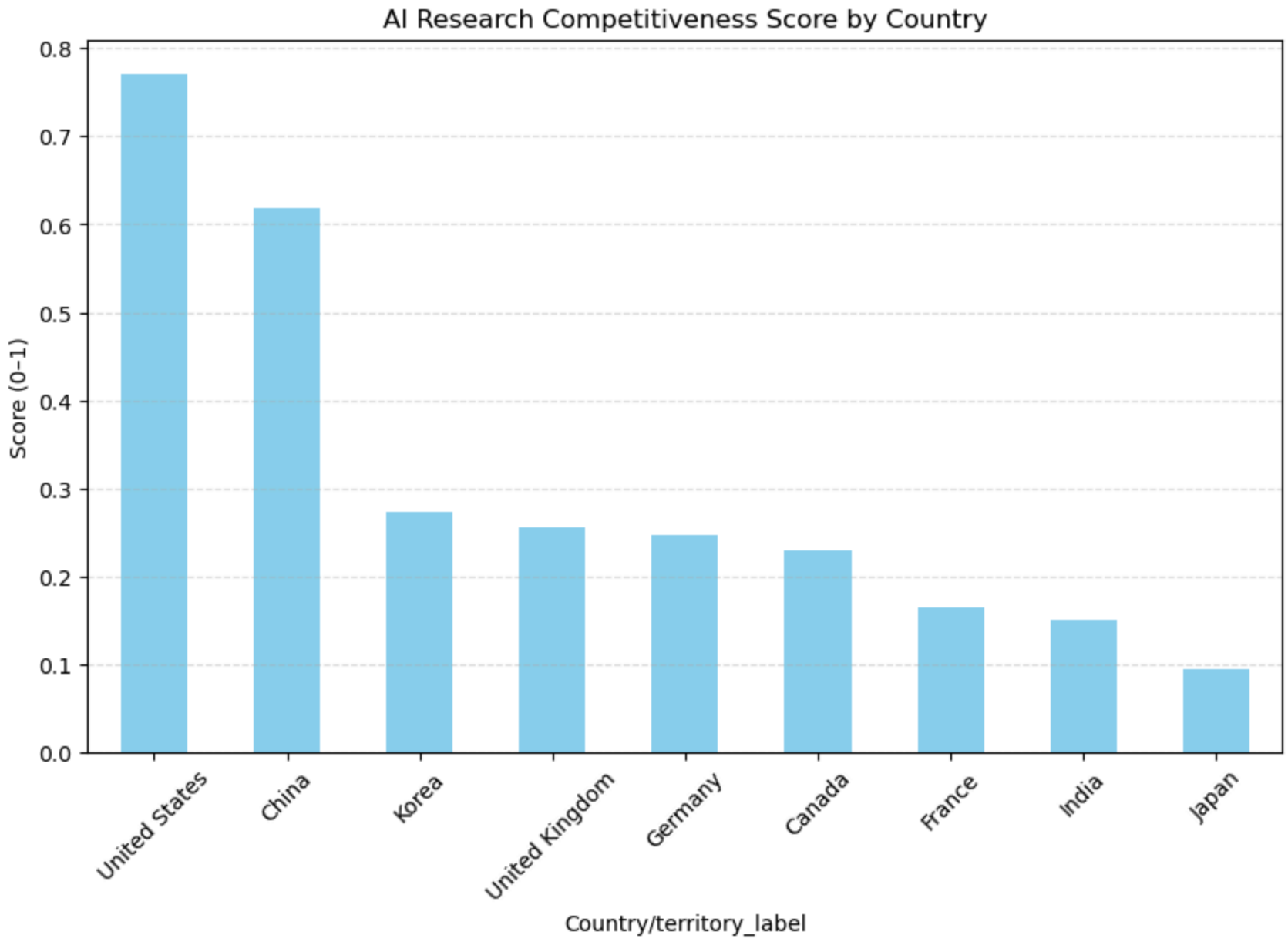
In [2]:

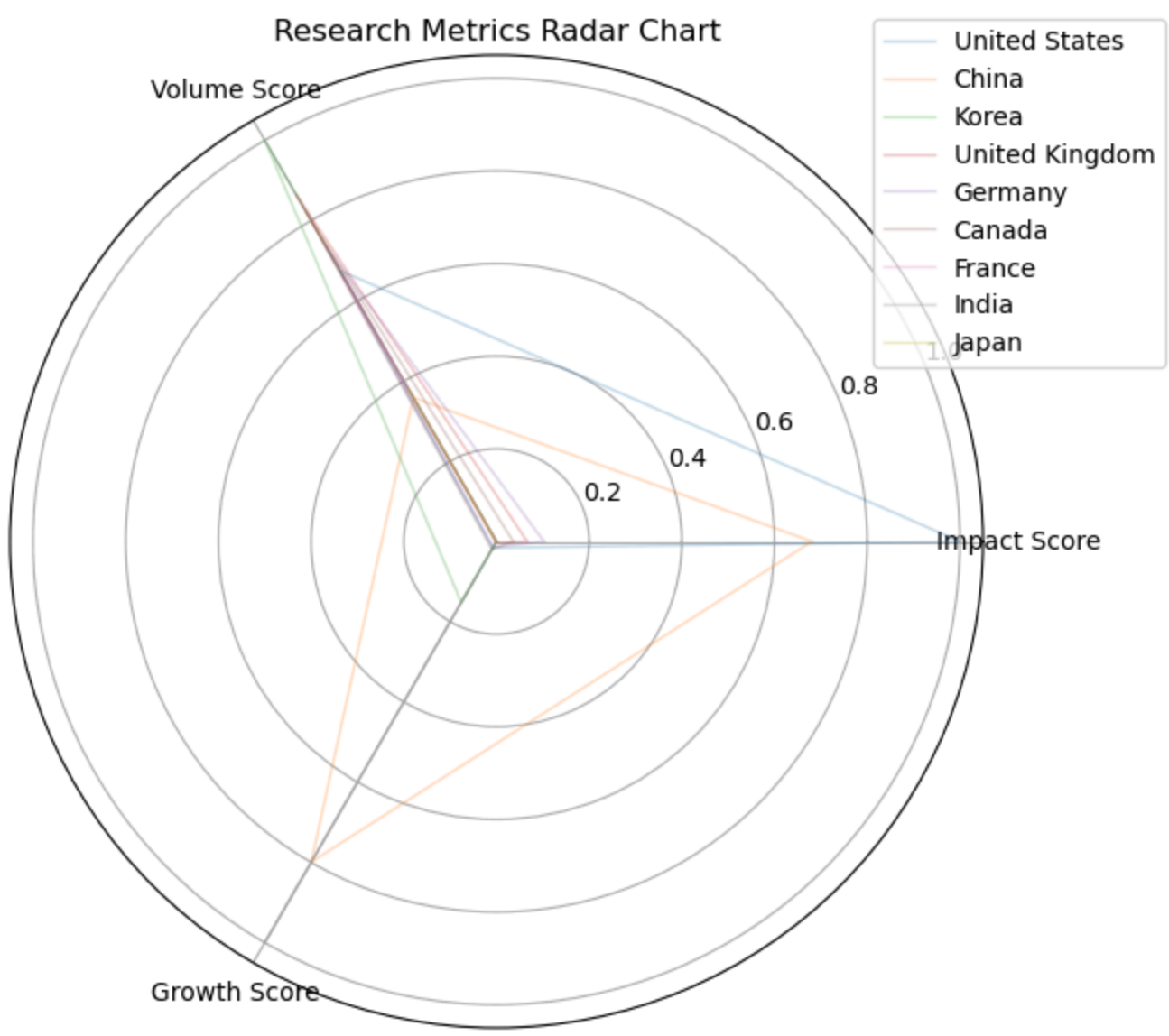
```
#Version 2

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ======================================================
# 1. Load Data
# ======================================================

df_AI_Article = pd.read_csv("1AI Article.csv")
df_pub_raw = pd.read_csv("1Top countries in AI publications in time.csv")

print("=== Impact Dataset Head ===")
print(df_AI_Article.head())

print("\n=== Publication Dataset Head ===")
print(df_pub_raw.head())


# ======================================================
# 2. Standardize Country Names
# ======================================================
```

```python
country_map = {
    "China (People's Republic of)": "China",
    "United States of America": "United States",
    "Republic of Korea": "Korea",
    "Korea, Rep.": "Korea",
    "Hong Kong": "China",
    "Taiwan": "China",
}

df_AI_Article["Geographic area"] = df_AI_Article["Geographic area"].replace(country_map)
df_pub_raw["Country/territory_label"] = df_pub_raw["Country/territory_label"].replace(country_map)


# ========================================================
# 3. Impact Score (High-impact research influence)
# ========================================================

# Groupby
df_imp = df_AI_Article.groupby("Geographic area")[
    "Number of highly cited publications in top 100"
].sum().reset_index()

df_imp["impact_share"] = (
    df_imp["Number of highly cited publications in top 100"]
    / df_imp["Number of highly cited publications in top 100"].sum()
)

impact_raw = df_imp.set_index("Geographic area")["impact_share"]

impact_score = (impact_raw - impact_raw.min()) / (impact_raw.max() - impact_raw.min())


# ========================================================
# 4. Publication Data Cleaning (Long → Wide)
# ========================================================

df_pub_filtered = df_pub_raw[
    (df_pub_raw["ai_concept"] == "All AI research") &
    (df_pub_raw["pub_type"] == "All publication types")
]

df_pub = df_pub_filtered.pivot_table(
    index="year",
    columns="Country/territory_label",
    values="publications",
    aggfunc="sum"
)

print("\n=== Pivoted Publication Table ===")
print(df_pub.head())


# ========================================================
# 5. Publication Volume Score (2021–2023) — Per Capita
# ========================================================

# Population  (2023)
population_dict = {
    "United States": 339996563,
    "China": 1425671352,
    "Canada": 40097761,
    "United Kingdom": 67736802,
    "France": 64756584,
    "Germany": 83294633,
    "Japan": 124127899,
    "Korea": 51784059,
    "India": 1428627663
}


df_pub_recent = df_pub.tail(3)

pub_volume_raw = df_pub_recent.mean()


pop = pub_volume_raw.index.map(lambda x: population_dict.get(x, np.nan))


pub_volume_pc = pub_volume_raw.values / pop


pub_volume_pc = pd.Series(pub_volume_pc, index=pub_volume_raw.index)


pub_volume_score = (pub_volume_pc - pub_volume_pc.min()) / (pub_volume_pc.max() - pub_volume_pc.min())


# ========================================================
# 6. Publication Growth Score (First → Last year)
# ========================================================

pub_growth_raw = (df_pub.iloc[-1] - df_pub.iloc[0]) / df_pub.iloc[0]
pub_growth_score = (pub_growth_raw - pub_growth_raw.min()) / (pub_growth_raw.max() - pub_growth_raw.min())


# ========================================================
```

```python
# 7. Align Countries for Scoring
# ======================================================

countries = pub_volume_score.index

impact_score = impact_score.reindex(countries, fill_value=0)
pub_growth_score = pub_growth_score.reindex(countries, fill_value=0)


# ======================================================
# 8. Compute Final Research Score
# ======================================================

research_score = (
    0.6 * impact_score +
    0.25 * pub_volume_score +
    0.15 * pub_growth_score
)

research_df = pd.DataFrame({
    "Impact Score": impact_score,
    "Volume Score": pub_volume_score,
    "Growth Score": pub_growth_score,
    "Research Score": research_score
}).sort_values("Research Score", ascending=False)

print("\n===== Final Research Score Table =====")
print(research_df)


# ======================================================
# 9. Visualization — Bar Chart
# ======================================================

plt.figure(figsize=(10, 6))
research_df["Research Score"].plot(kind="bar", color="skyblue")
plt.title("AI Research Competitiveness Score by Country")
plt.ylabel("Score (0–1)")
plt.xticks(rotation=45)
plt.grid(axis="y", linestyle="--", alpha=0.4)
plt.show()


# ======================================================
# 10. Visualization — Radar Chart
# ======================================================

from math import pi

metrics = ["Impact Score", "Volume Score", "Growth Score"]
N = len(metrics)

plt.figure(figsize=(7, 7))
angles = [n / float(N) * 2 * pi for n in range(N)]
angles += angles[:1]

for country in research_df.index:
    values = research_df.loc[country, metrics].tolist()
    values += values[:1]
    plt.polar(angles, values, linewidth=1, alpha=0.25, label=country)

plt.xticks(angles[:-1], metrics)
plt.title("Research Metrics Radar Chart")
plt.legend(bbox_to_anchor=(1.2, 1.05))
plt.show()

# ============================================
# 11. Export Research Score as Dictionary
# ============================================

df_research = research_df["Research Score"].to_dict()

print("\n=== Research Score Dictionary ===")
print(df_research)
```

```
=== Impact Dataset Head ===
   Number of highly cited publications in top 100 Geographic area  Label
0                                              50   United States   2023
1                                              34           China   2023
2                                               7         Germany   2023
3                                               7       Hong Kong   2023
4                                               6          Canada   2023

=== Publication Dataset Head ===
        ai_concept Country/territory       Country/territory_label  \
0  All AI research               CHN  China (People's Republic of)
1  All AI research               CHN  China (People's Republic of)
2  All AI research               CHN  China (People's Republic of)
3  All AI research               CHN  China (People's Republic of)
4  All AI research               CHN  China (People's Republic of)

  impact_level               pub_type  publications  year
0    All works  All publication types    638.583334  2000
1    All works  All publication types    831.100001  2001
2    All works  All publication types   3143.697621  2002
3    All works  All publication types   2822.383338  2003
4    All works  All publication types   4152.416669  2004

=== Pivoted Publication Table ===
Country/territory_label       Canada        China       France      Germany  \
year
2000                     434.798486   638.583334   633.779366   842.194662
2001                     424.350000   831.100001   511.955557   873.214287
2002                    1854.115084  3143.697621  1846.916887  2422.616814
2003                     999.511113  2822.383338  1102.881820  1259.405557
2004                     813.142859  4152.416669  1087.210175  1289.352311

Country/territory_label       India        Japan        Korea  United Kingdom  \
year
2000                    154.179798  1162.583334   239.833334     1036.065045
2001                    144.166667  1109.333334   233.133334     1057.722224
2002                    559.782938  5140.350940  1019.116668     2721.166346
2003                    388.666667  2986.093420   597.350001     1826.264685
2004                    471.875001  2502.774173   634.883334     1520.409744

Country/territory_label  United States
year
2000                       3628.998705
2001                       3389.037307
2002                      14443.905785
2003                       8505.192256
2004                       5870.007980

===== Final Research Score Table =====
                         Impact Score  Volume Score  Growth Score  \
Country/territory_label
United States                1.000000      0.675423      0.015975
China                        0.680723      0.358840      0.799173
Korea                        0.000000      1.000000      0.151776
United Kingdom               0.066265      0.864780      0.002379
Germany                      0.102410      0.733607      0.014253
Canada                       0.036145      0.821576      0.019863
France                       0.000000      0.655558      0.000944
India                        0.000000      0.000000      1.000000
Japan                        0.000000      0.379326      0.000000

                         Research Score
Country/territory_label
United States                  0.771252
China                          0.618020
Korea                          0.272766
United Kingdom                 0.256311
Germany                        0.246985
Canada                         0.230060
France                         0.164031
India                          0.150000
Japan                          0.094831
```






```
=== Research Score Dictionary ===
{'United States': 0.7712519660242639, 'China': 0.6180195472681431, 'Korea': 0.2727663299264079, 'United Kingdom': 0.256310775855620
9, 'Germany': 0.24698535607933575, 'Canada': 0.2300600164719307, 'France': 0.16403105909300955, 'India': 0.15, 'Japan': 0.094831432
28122126}
```

In [3]:
```
# Export
research_df.to_csv("Research_Score.csv", encoding="utf-8-sig")
```

2 Talent

In [4]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# ========================
# Country column
# ========================
def find_country_col(df):
    for key in ["geo", "country", "area", "region"]:
        match = [c for c in df.columns if key in c.lower()]
        if match:
```

```python
            return match[0]
    raise ValueError("❌ can't find the country")

# find indicator
def find_col(df, keyword):
    match = [c for c in df.columns if keyword.lower() in c.lower()]
    if match:
        return match[0]
    raise ValueError(f"❌ can't find the column including {keyword}")

# %
def clean_percent(series):
    return (series.astype(str)
                  .str.replace("%","")
                  .str.replace(",","")
                  .replace("", np.nan)
                  .astype(float))

# making countries into index
def norm_country(df):
    c = find_country_col(df)
    df[c] = df[c].str.strip()
    return df.set_index(c)

# ===== get the newest row of the country =====
def get_latest_per_country(df):
    country_col = find_col(df, "geo")
    year_cols = [c for c in df.columns if "year" in c.lower()]

    if len(year_cols) > 0:
        year_col = year_cols[0]
        df_sorted = df.sort_values(year_col)
        return df_sorted.groupby(country_col).tail(1).set_index(country_col)
    else:
        return df.groupby(country_col).mean().set_index(country_col)


# ========================
# Data
# ========================
df_conc       = pd.read_csv("2 AI concentration.csv")
df_concchg    = pd.read_csv("2 %cahnge in AI concentration.csv")
df_pen        = pd.read_csv("2 AI Penetration.csv")
df_repr       = pd.read_csv("2 AI talent representation.csv")
df_migr       = pd.read_csv("2 AI talent migration.csv")

df_jobpost    = pd.read_csv("2 AI job posting.csv")
df_hiring     = pd.read_csv("2 AI hiring rate.csv")

df_robot      = pd.read_csv("2 Number of Industrial robots installed.csv")
df_robot_g    = pd.read_csv("2 Growth rate of Industrial robots.csv")

df_inv24      = pd.read_csv("2 Investment in AI yr2024.csv")
df_inv_total  = pd.read_csv("2 Total AI investment 2013-2024.csv")
df_new_ai     = pd.read_csv("2 New funded AI companies 2024.csv")
df_new_ai12   = pd.read_csv("2 New funded AI companies 2013-2024.csv")

# ========================
# Stock: number of AI talent
# ========================
stock = pd.DataFrame()

stock["concentration"] = clean_percent(
    norm_country(df_conc)[find_col(df_conc,"concentration")]
)

stock["concentration_change"] = clean_percent(
    norm_country(df_concchg)[find_col(df_concchg,"change")]
)

stock["penetration"] = clean_percent(
    norm_country(df_pen)[find_col(df_pen,"penetration")]
)

# ======== recognize the column ========
def find_col(df, keyword):
    return [c for c in df.columns if keyword.lower() in c.lower()][0]

def clean_percent(series):
    return series.astype(str).str.replace("%", "").astype(float)

# ======== Deal with AI talent representation (newest year) ========

# 1. find column
country_col = find_col(df_repr, "geo")     # Geographic area
year_col    = find_col(df_repr, "year")    # Year
repr_col    = find_col(df_repr, "repr")    # Representation

# 2. read the newest row
df_repr_latest = (
    df_repr.sort_values(year_col)
           .groupby(country_col)
           .tail(1)
           .set_index(country_col)
)
```

```python
# 3. %
df_repr_latest["representation_clean"] = clean_percent(df_repr_latest[repr_col])

# 4. Save to Talent Stock
stock["representation"] = df_repr_latest["representation_clean"]


# ========================
# Attractiveness
# ========================
attr = pd.DataFrame()

attr["migration"] = clean_percent(
    norm_country(df_migr)[find_col(df_migr,"migration")]
)

df_jobpost_latest = get_latest_per_country(df_jobpost)

attr["job_posting"] = clean_percent(
    df_jobpost_latest[find_col(df_jobpost, "posting")]
)

attr["hiring_rate"] = clean_percent(
    norm_country(df_hiring)[find_col(df_hiring,"hiring")]
)


# ========================
# Ecosystem
# ========================
eco = pd.DataFrame()

# number of robots
robot_col = find_col(df_robot, "robot")
eco["robots"] = norm_country(df_robot)[robot_col].astype(float) * 1000

# growth rate of robots
eco["robot_growth"] = clean_percent(
    norm_country(df_robot_g)[find_col(df_robot_g, "growth")]
)

eco["AI_invest_2024"] = norm_country(df_inv24)[find_col(df_inv24,"invest")].astype(float)

eco["AI_total_invest"] = norm_country(df_inv_total)[find_col(df_inv_total,"invest")].astype(float)

eco["new_AI_startups"] = norm_country(df_new_ai)[find_col(df_new_ai,"fund")].astype(float)
eco["new_AI_startups_10yr"] = norm_country(df_new_ai12)[find_col(df_new_ai12,"fund")].astype(float)

# ========================
# combine + PCA
# ========================
talent = pd.concat([stock, attr, eco], axis=1)
talent = talent.dropna(how="all")

# standardize
talent_std = talent.fillna(0)
scaler = StandardScaler()
talent_scaled = scaler.fit_transform(talent_std)


pca = PCA(n_components=1)
talent["Talent Score"] = pca.fit_transform(talent_scaled)


talent["Talent Score"] = (talent["Talent Score"] - talent["Talent Score"].min()) / (
    talent["Talent Score"].max() - talent["Talent Score"].min()
)

# ========================
# Canada vs target countries
# ========================
target = [
    "Canada","United States","China","Germany","South Korea","United Kingdom",
    "France","India","Japan"
]

print("\n=== 🎯 Talent PCA Score (Canada vs World) ===")
print(talent.loc[talent.index.intersection(target)].sort_values("Talent Score", ascending=False))



# ====================================================
# Export Final Talent Scores (sorted + target only)
# ====================================================


talent_sorted = talent.sort_values("Talent Score", ascending=False)


talent_target_sorted = talent_sorted[talent_sorted.index.isin(target)]


talent_target_sorted.to_csv("Talent_Target_Score.csv", encoding="utf-8-sig")
```

```
print("CSV saved: Talent_Target_Score.csv (sorted descending)")
print(talent_target_sorted)
```

```
=== 🎯 Talent PCA Score (Canada vs World) ===
                 concentration  concentration_change  penetration  \
Geographic area
United States             0.78                 127.0         2.63
China                      NaN                   NaN          NaN
India                     0.87                 252.0         2.51
United Kingdom            0.87                 109.0         1.40
Canada                    0.93                 166.0         1.30
Germany                   1.09                 140.0         1.32
France                    0.79                  52.0         1.23
South Korea               1.07                  65.0         0.63
Japan                      NaN                   NaN          NaN

                 representation  migration  job_posting  hiring_rate  \
Geographic area
United States             33.68       1.07         1.79        24.73
China                       NaN        NaN          NaN          NaN
India                     70.08      -1.55          NaN        33.39
United Kingdom            70.50       0.55         1.26        13.78
Canada                    32.22       0.95         1.41        26.13
Germany                   23.77       2.13         1.15         9.10
France                    68.95       0.34         1.10         5.75
South Korea                 NaN      -0.36          NaN        13.21
Japan                       NaN        NaN          NaN          NaN

                   robots  robot_growth  AI_invest_2024  AI_total_invest  \
Geographic area
United States     37600.0          -5.0      109.077326       470.922311
China            276300.0          -5.0        9.290338       119.319823
India              8500.0          59.0        1.160556        11.294863
United Kingdom     3800.0          51.0        4.519300        28.170251
Canada             4300.0          37.0        2.887230        15.305995
Germany           28400.0           7.0        1.974363        13.270770
France             6400.0         -13.0        2.617394        11.096808
South Korea       31400.0          -1.0        1.326996         8.962057
Japan             46100.0          -9.0        0.927028         5.892678

                 new_AI_startups  new_AI_startups_10yr  Talent Score
Geographic area
United States             1073.0                6956.0      1.000000
China                       98.0                1605.0      0.194811
India                       74.0                 434.0      0.183520
United Kingdom             116.0                 885.0      0.180254
Canada                      51.0                 481.0      0.149036
Germany                     67.0                 394.0      0.131884
France                      59.0                 468.0      0.115731
South Korea                 52.0                 270.0      0.077766
Japan                       42.0                 388.0      0.037944
CSV saved: Talent_Target_Score.csv (sorted descending)
                 concentration  concentration_change  penetration  \
Geographic area
United States             0.78                 127.0         2.63
China                      NaN                   NaN          NaN
India                     0.87                 252.0         2.51
United Kingdom            0.87                 109.0         1.40
Canada                    0.93                 166.0         1.30
Germany                   1.09                 140.0         1.32
France                    0.79                  52.0         1.23
South Korea               1.07                  65.0         0.63
Japan                      NaN                   NaN          NaN

                 representation  migration  job_posting  hiring_rate  \
Geographic area
United States             33.68       1.07         1.79        24.73
China                       NaN        NaN          NaN          NaN
India                     70.08      -1.55          NaN        33.39
United Kingdom            70.50       0.55         1.26        13.78
Canada                    32.22       0.95         1.41        26.13
Germany                   23.77       2.13         1.15         9.10
France                    68.95       0.34         1.10         5.75
South Korea                 NaN      -0.36          NaN        13.21
Japan                       NaN        NaN          NaN          NaN

                   robots  robot_growth  AI_invest_2024  AI_total_invest  \
Geographic area
United States     37600.0          -5.0      109.077326       470.922311
China            276300.0          -5.0        9.290338       119.319823
India              8500.0          59.0        1.160556        11.294863
United Kingdom     3800.0          51.0        4.519300        28.170251
Canada             4300.0          37.0        2.887230        15.305995
Germany           28400.0           7.0        1.974363        13.270770
France             6400.0         -13.0        2.617394        11.096808
South Korea       31400.0          -1.0        1.326996         8.962057
Japan             46100.0          -9.0        0.927028         5.892678

                 new_AI_startups  new_AI_startups_10yr  Talent Score
Geographic area
United States             1073.0                6956.0      1.000000
China                       98.0                1605.0      0.194811
India                       74.0                 434.0      0.183520
United Kingdom             116.0                 885.0      0.180254
Canada                      51.0                 481.0      0.149036
Germany                     67.0                 394.0      0.131884
France                      59.0                 468.0      0.115731
South Korea                 52.0                 270.0      0.077766
Japan                       42.0                 388.0      0.037944
```

This script constructs a data-driven AI Talent Competitiveness Index by combining multiple datasets describing three major dimensions of talent strength: (1) Stock (e.g., AI concentration, penetration, representation), (2) Attractiveness (e.g., migration flows, job postings, hiring rate), and (3) Ecosystem (e.g., industrial robots, AI investment, startup formation).

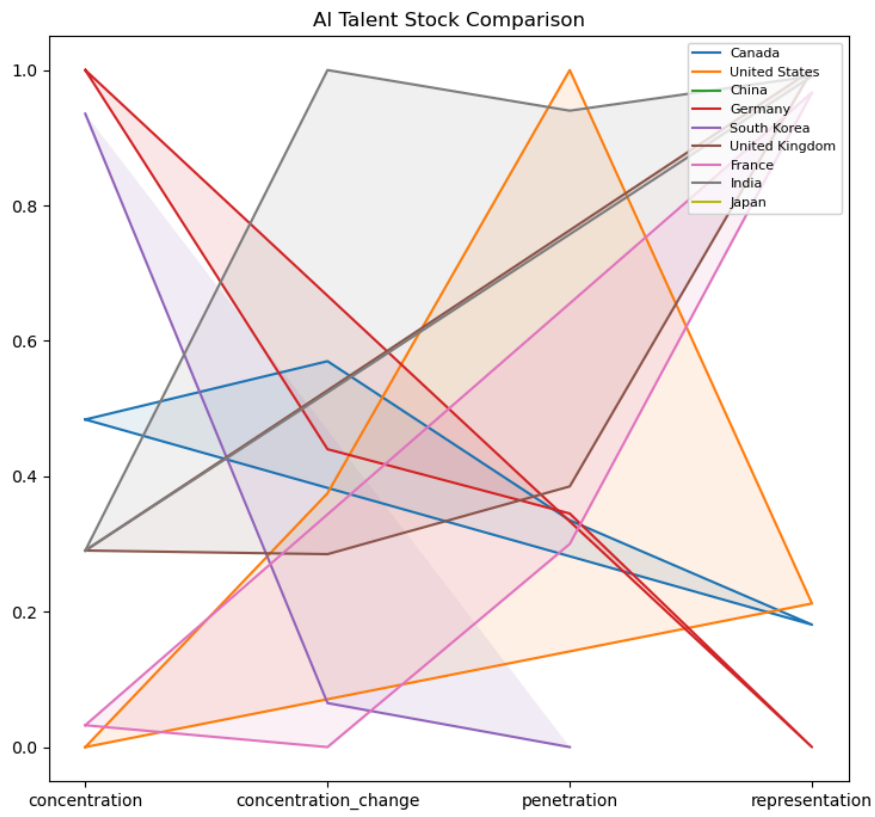
The code automatically detects country columns, cleans percentage values, unifies country names, and extracts each country's latest available data. All indicators are merged into a single table, missing values are filled with 0, and the dataset is standardized. A PCA model is then applied to extract the first principal component, which serves as a composite score representing overall AI talent competitiveness. The score is normalized to a 0–1 range and used to compare target countries such as Canada, China, the U.S., and others.

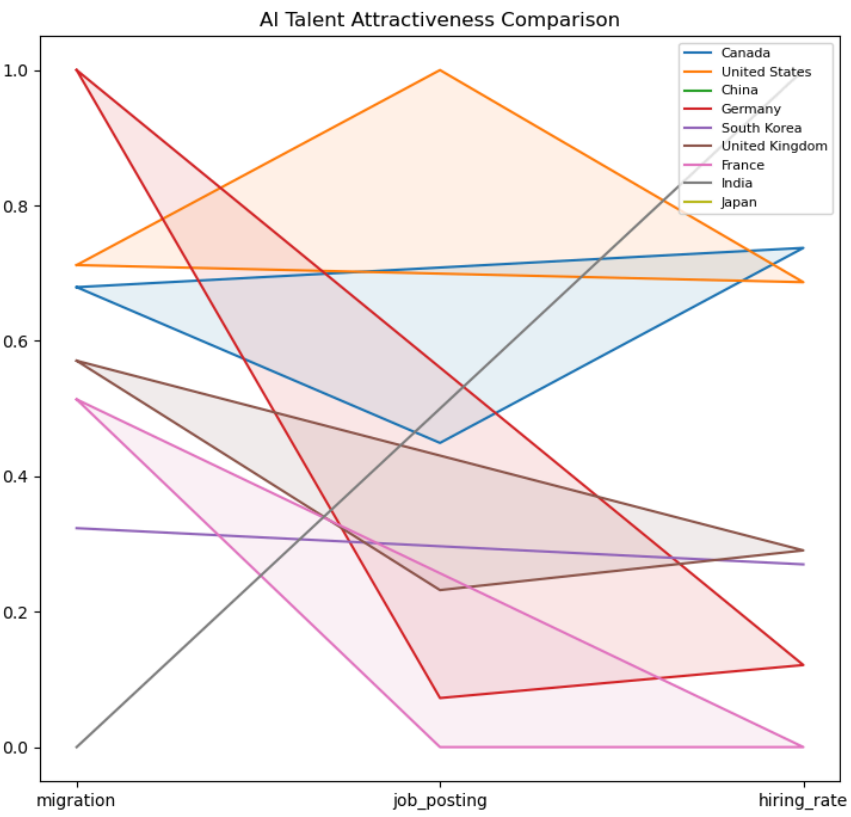
The final Talent Score is derived using Principal Component Analysis (PCA). After all talent indicators are merged, cleaned, and standardized, PCA is applied to the full feature matrix. The first principal component (PC1) captures the largest share of variance across all indicators and therefore represents the strongest, data-driven direction of variation in countries' AI talent capacity.

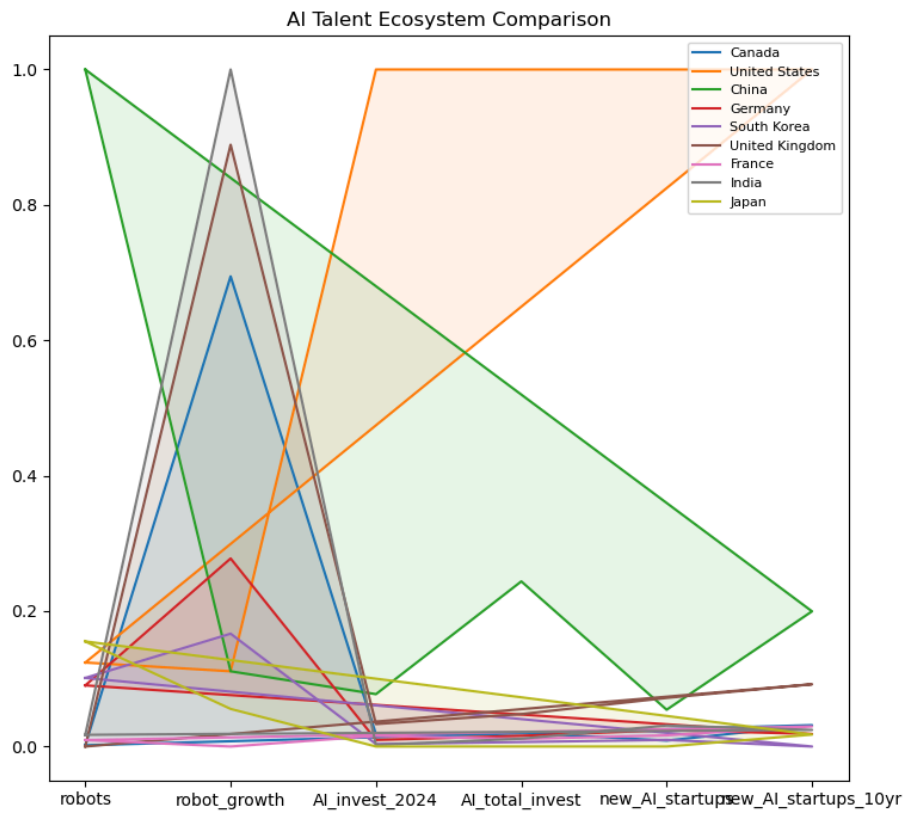
PC1 is used as the overall talent competitiveness index because it assigns automatically learned weights to each indicator, reflecting their empirical importance in explaining cross-country differences. The raw PC1 values are then min-max normalized to a 0–1 range so that higher scores consistently indicate stronger AI talent competitiveness.

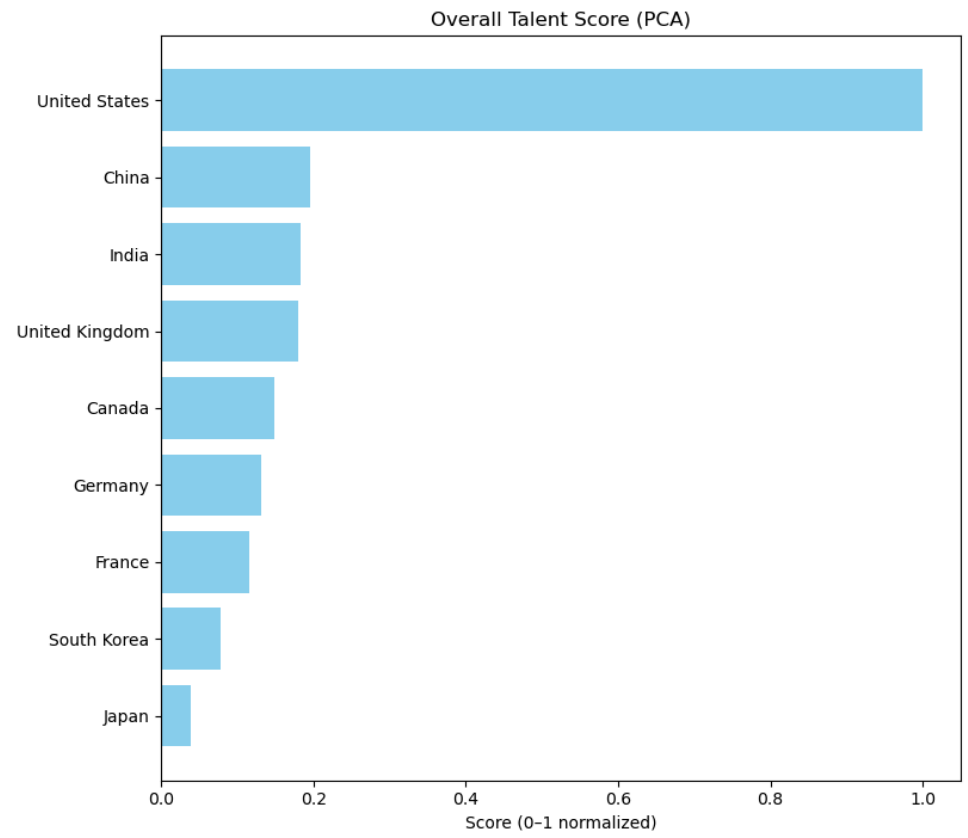
In [5]:

```
# graph
import matplotlib.pyplot as plt
import numpy as np

# ==========================================================================================
#  Data preparing (4 dimension)
# ==========================================================================================

# 1) Stock
stock_cols = ["concentration", "concentration_change", "penetration", "representation"]
stock_df = talent.loc[target, stock_cols]

# 2) Attractiveness
attr_cols = ["migration", "job_posting", "hiring_rate"]
attr_df = talent.loc[target, attr_cols]

# 3) Ecosystem
eco_cols = ["robots", "robot_growth", "AI_invest_2024", "AI_total_invest",
            "new_AI_startups", "new_AI_startups_10yr"]
eco_df = talent.loc[target, eco_cols]

# 4) Talent Score
score_df = talent.loc[target, ["Talent Score"]].sort_values("Talent Score", ascending=True)


# ==========================================================================================
#  Radar graph
# ==========================================================================================
def plot_radar(df, title, ax):
    categories = df.columns.tolist()
    N = len(categories)

    # Angel
    angles = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()
    df_norm = (df - df.min()) / (df.max() - df.min())

    for country in df_norm.index:
        values = df_norm.loc[country].tolist()
        values += values[:1]
        ax.plot(angles + angles[:1], values, label=country)
        ax.fill(angles + angles[:1], values, alpha=0.1)

    ax.set_title(title, size=12)
    ax.set_xticks(angles)
    ax.set_xticklabels(categories)
    ax.legend(fontsize=8, loc='upper right')

# ==========================================================================================
#  Draw 4 graphes
# ==========================================================================================
fig, axs = plt.subplots(2, 2, figsize=(16, 14))
axs = axs.flatten()

# ---- subplot 1 ----
plot_radar(stock_df, "AI Talent Stock Comparison", axs[0])

# ---- subplot 2 ----

plot_radar(attr_df, "AI Talent Attractiveness Comparison", axs[1])

# ---- subplot 3 ----
plot_radar(eco_df, "AI Talent Ecosystem Comparison", axs[2])

# ---- subplot 4: Talent Score ----
axs[3].barh(score_df.index, score_df["Talent Score"], color='skyblue')
axs[3].set_title("Overall Talent Score (PCA)", fontsize=12)
axs[3].set_xlabel("Score (0–1 normalized)")

plt.tight_layout()
plt.show()
```









These plots compare Canada with major AI countries across four dimensions. The Talent Stock radar chart shows Canada performing at a mid-high level in AI talent concentration and skill penetration, though below the United States and the UK. The Attractiveness chart indicates that Canada maintains balanced talent migration and steady hiring demand, stronger than countries like France or Japan but weaker than the U.S. In the Ecosystem comparison, Canada's AI investment, robotics adoption, and startup activity place it in the global middle tier—solid but much smaller in scale than the United States or China. Finally, the PCA Overall Talent Score summarizes all metrics: Canada ranks in the "second tier," behind the U.S. and China but ahead of countries like France, Japan, and South Korea.

3 Compute

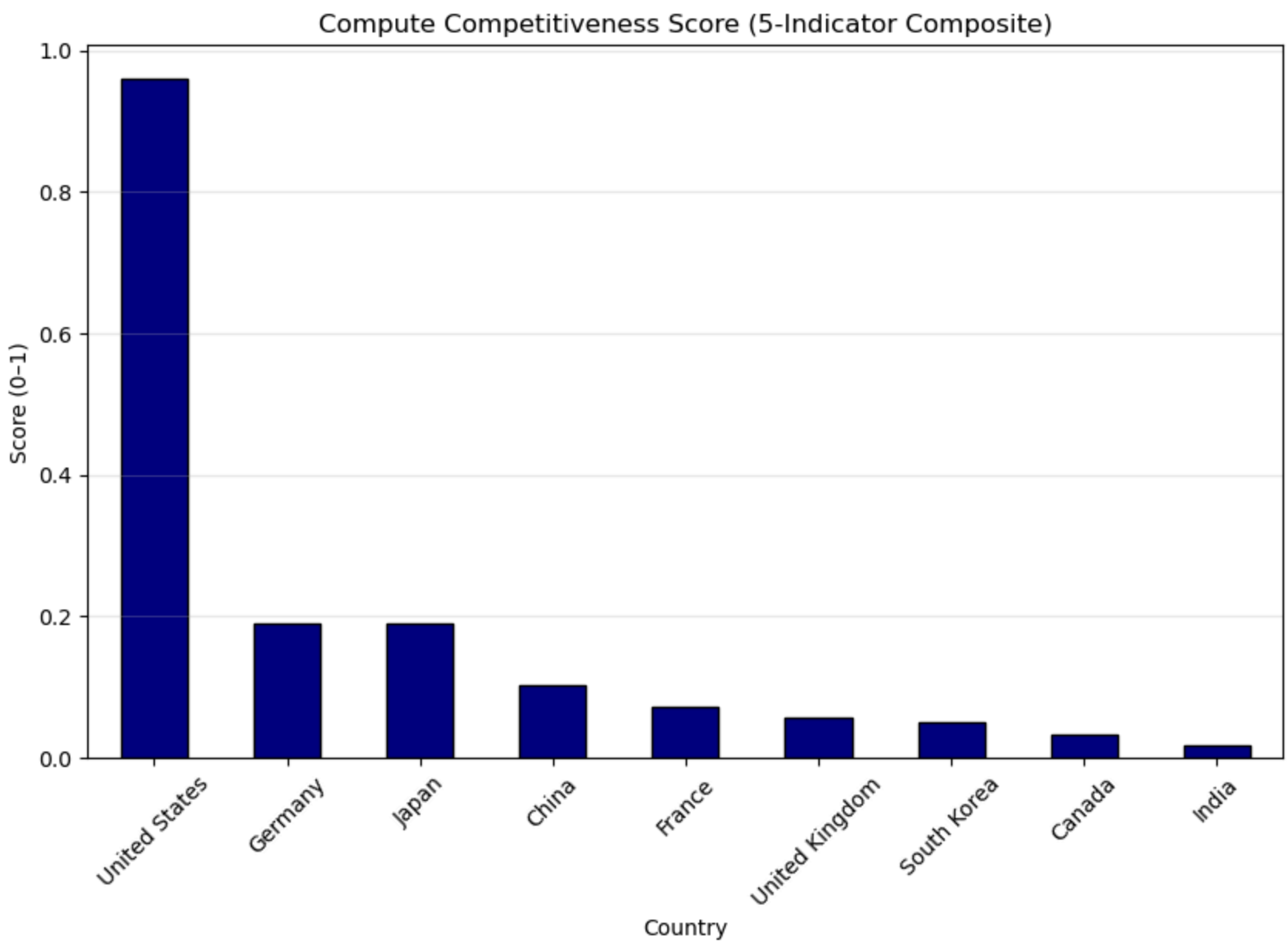
In [6]:
```
# Compute worldwide

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ============================
# 1. Load dataset
# ============================

df = pd.read_csv("3 TOP500Calculation.csv")

# ============================
# 2. Standardize Country Names
# ============================

country_map = {
    "Taiwan": "China",
    "Taiwan, China": "China",
    "Hong Kong": "China",
    "Hong Kong SAR": "China",
    "Hong Kong S.A.R.": "China",
}

df["Country"] = df["Country"].replace(country_map)

# ============================
# 3. Clean numeric columns
# ============================

numeric_cols = [
    "Rmax [TFlop/s]",
    "Total Cores",
```

```python
    "Accelerator/Co-Processor Cores",
    "Energy Efficiency [GFlops/Watts]"
]

for col in numeric_cols:
    df[col] = (
        df[col]
        .astype(str)
        .str.replace(",", "", regex=False)
        .str.extract(r"(\d+\.?\d*)")[0]
        .astype(float)
    )

df[numeric_cols] = df[numeric_cols].fillna(0)

# Count system number
df["SystemCount"] = 1

# ============================
# 4. Aggregate by Country
# ============================

compute_raw = df.groupby("Country").agg({
    "Rmax [TFlop/s]": "sum",
    "Total Cores": "sum",
    "Accelerator/Co-Processor Cores": "sum",
    "Energy Efficiency [GFlops/Watts]": "mean",
    "SystemCount": "sum"
})

# ============================
# 5. Normalize all 5 metrics
# ============================

compute_norm = (compute_raw - compute_raw.min()) / (compute_raw.max() - compute_raw.min())

compute_norm.rename(columns={
    "Rmax [TFlop/s]": "Rmax_score",
    "Total Cores": "Cores_score",
    "Accelerator/Co-Processor Cores": "GPU_score",
    "Energy Efficiency [GFlops/Watts]": "Energy_score",
    "SystemCount": "Count_score"
}, inplace=True)

# ============================
# 6. Final Compute Score (Weighted)
# ============================

compute_norm["Compute Score"] = (
    0.50 * compute_norm["Rmax_score"] +
    0.20 * compute_norm["Cores_score"] +
    0.20 * compute_norm["GPU_score"] +
    0.05 * compute_norm["Energy_score"] +
    0.05 * compute_norm["Count_score"]
)

compute_score = compute_norm.sort_values("Compute Score", ascending=False)

print("\n=== Final Compute Score Ranking ===")
print(compute_score)


# ============================
# 7. Filter target countries
# ============================

target_countries = [
    "Canada", "United States", "China",
    "Germany", "South Korea", "United Kingdom",
    "France", "India", "Japan"
]

compute_target = compute_score.loc[compute_score.index.intersection(target_countries)]
print("\n=== Target Country Compute Score ===")
print(compute_target)


# ============================
# 8. Bar Chart
# ============================

plt.figure(figsize=(10,6))
compute_target["Compute Score"].plot(kind="bar", color="navy", edgecolor="black")
plt.title("Compute Competitiveness Score (5-Indicator Composite)")
plt.ylabel("Score (0–1)")
plt.xticks(rotation=45)
plt.grid(axis="y", alpha=0.3)
plt.show()

# ============================================
# 9. Export Compute Scores as Dictionary
# ============================================

df_compute = compute_target["Compute Score"].to_dict()

print("\n=== Compute Score Dictionary (Target Countries) ===")
```

```python
print(df_compute)


# ==================================================
# 10. Export Compute Scores to CSV
# ==================================================

# 1. Full table
compute_score.to_csv("Compute_Full_Score.csv", encoding="utf-8-sig")

# 2. Target country
compute_target.to_csv("Compute_Target_Score.csv", encoding="utf-8-sig")

print("CSV files saved: Compute_Full_Score.csv + Compute_Target_Score.csv")
```

```
=== Final Compute Score Ranking ===
                     Rmax_score  Cores_score  GPU_score  Energy_score  \
Country
United States          1.000000     1.000000   1.000000      0.198550
Germany                0.199929     0.149297   0.122409      0.494395
Japan                  0.204214     0.230384   0.079125      0.248933
Italy                  0.127136     0.130404   0.144213      0.249577
China                  0.058571     0.247308   0.021362      0.084880
Switzerland            0.064214     0.038580   0.036080      1.000000
Finland                0.055761     0.052440   0.065331      0.496916
France                 0.045604     0.074726   0.033314      0.404790
United Kingdom         0.054867     0.046909   0.030915      0.235545
Netherlands            0.041401     0.023393   0.026873      0.481436
Denmark                0.009157     0.003388   0.005135      0.889531
South Korea            0.045965     0.047057   0.037226      0.103615
Spain                  0.031438     0.025744   0.015056      0.414396
Saudi Arabia           0.030669     0.050624   0.027514      0.230028
Brazil                 0.017468     0.019250   0.024881      0.342533
Hungary                0.000047     0.000078   0.000616      0.659770
Luxembourg             0.001111     0.001288   0.002200      0.631431
Canada                 0.022304     0.033341   0.009053      0.149747
Slovakia               0.003081     0.001180   0.001721      0.598290
Belgium                0.000000     0.000000   0.000550      0.626044
Sweden                 0.009210     0.007255   0.005133      0.440784
Turkey                 0.001768     0.000997   0.000562      0.592903
United Arab Emirates   0.019943     0.014496   0.020711      0.222219
Australia              0.007526     0.008015   0.008159      0.333633
Poland                 0.009787     0.007065   0.006127      0.209208
Thailand               0.002758     0.001889   0.003011      0.307284
India                  0.015887     0.016491   0.008767      0.045632
Norway                 0.005926     0.010498   0.002500      0.156843
Czechia                0.002183     0.005712   0.001584      0.213522
Portugal               0.000178     0.000935   0.000000      0.253181
Israel                 0.015215     0.006318   0.009229      0.000000
Russia                 0.009499     0.010745   0.014806      0.000000
Morocco                0.000055     0.000814   0.000000      0.097900
Singapore              0.004726     0.003287   0.002793      0.000000
Vietnam                0.006295     0.002018   0.003414      0.000000
Kazakhstan             0.005113     0.001716   0.002904      0.000000
Austria                0.004561     0.002983   0.002581      0.000000
Slovenia               0.000594     0.002259   0.000660      0.000000
Ireland                0.000563     0.002189   0.000000      0.000000
Iceland                0.001113     0.000220   0.000833      0.000000
Bulgaria               0.000250     0.002054   0.000000      0.000000
Argentina              0.000375     0.000336   0.000965      0.000000

                     Count_score  Compute Score
Country
United States           1.000000       0.959928
Germany                 0.228070       0.190429
Japan                   0.245614       0.188736
Italy                   0.099415       0.135941
China                   0.280702       0.101299
Switzerland             0.011696       0.097624
Finland                 0.011696       0.076865
France                  0.122807       0.070790
United Kingdom          0.052632       0.057407
Netherlands             0.035088       0.056580
Denmark                 0.000000       0.050759
South Korea             0.081871       0.049113
Spain                   0.011696       0.045183
Saudi Arabia            0.035088       0.044218
Brazil                  0.052632       0.037318
Hungary                 0.000000       0.033151
Luxembourg              0.000000       0.032825
Canada                  0.105263       0.032381
Slovakia                0.005848       0.032328
Belgium                 0.000000       0.031412
Sweden                  0.040936       0.031168
Turkey                  0.005848       0.031133
United Arab Emirates    0.023392       0.029293
Australia               0.017544       0.024557
Poland                  0.040936       0.020039
Thailand                0.005848       0.018016
India                   0.029240       0.016739
Norway                  0.046784       0.015744
Czechia                 0.011696       0.013812
Portugal                0.000000       0.012935
Israel                  0.011696       0.011302
Russia                  0.023392       0.011029
Morocco                 0.000000       0.005085
Singapore               0.017544       0.004456
Vietnam                 0.000000       0.004234
Kazakhstan              0.005848       0.003773
Austria                 0.005848       0.003686
Slovenia                0.005848       0.001173
Ireland                 0.005848       0.001011
Iceland                 0.000000       0.000767
Bulgaria                0.000000       0.000536
Argentina               0.000000       0.000448

=== Target Country Compute Score ===
                Rmax_score  Cores_score  GPU_score  Energy_score  Count_score  \
Country
United States     1.000000     1.000000   1.000000      0.198550     1.000000
Germany           0.199929     0.149297   0.122409      0.494395     0.228070
```

```
Japan            0.204214    0.230384   0.079125      0.248933     0.245614
China            0.058571    0.247308   0.021362      0.084880     0.280702
France           0.045604    0.074726   0.033314      0.404790     0.122807
United Kingdom   0.054867    0.046909   0.030915      0.235545     0.052632
South Korea      0.045965    0.047057   0.037226      0.103615     0.081871
Canada           0.022304    0.033341   0.009053      0.149747     0.105263
India            0.015887    0.016491   0.008767      0.045632     0.029240

               Compute Score
Country
United States       0.959928
Germany             0.190429
Japan               0.188736
China               0.101299
France              0.070790
United Kingdom      0.057407
South Korea         0.049113
Canada              0.032381
India               0.016739
```



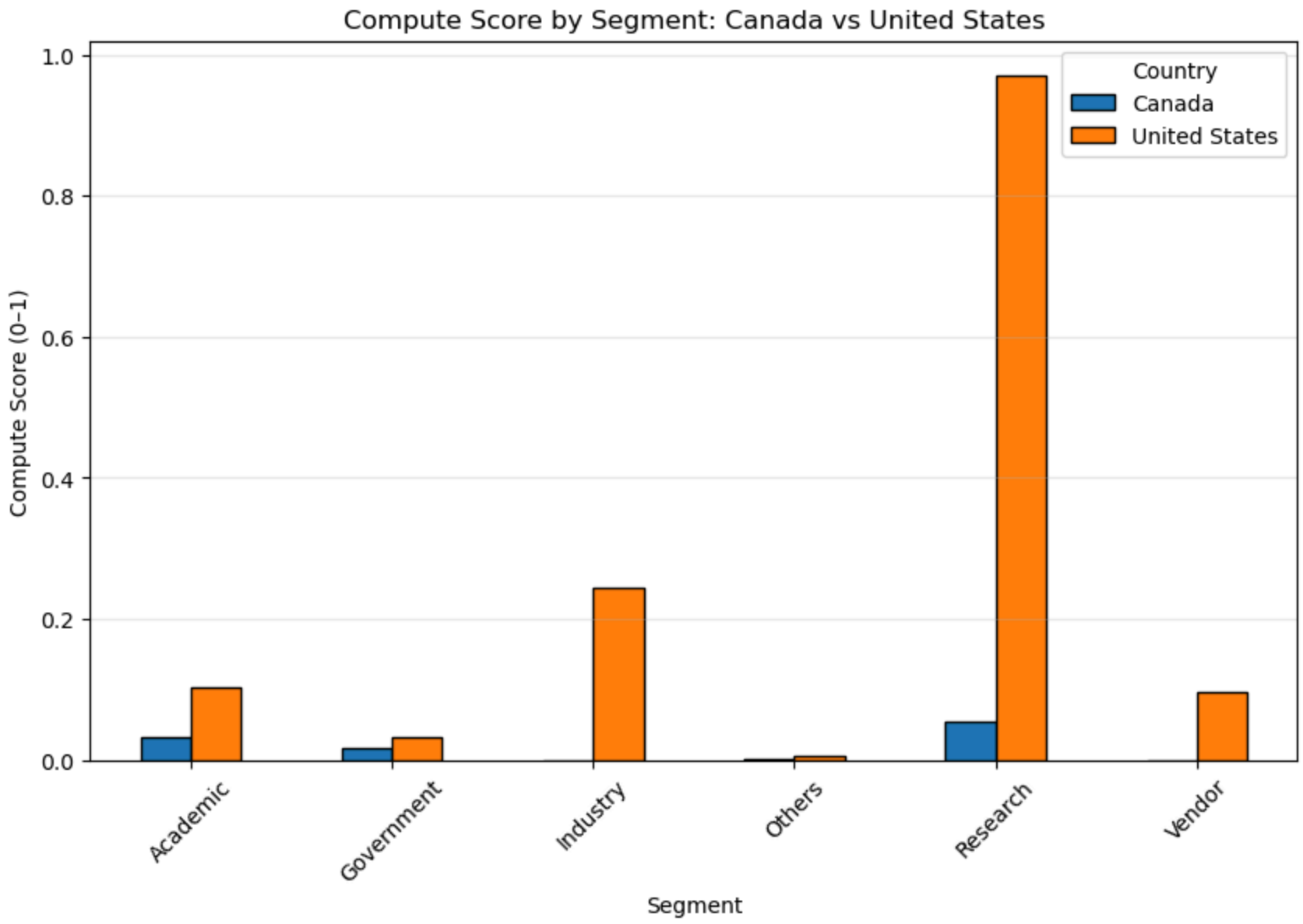

```
=== Compute Score Dictionary (Target Countries) ===
{'United States': 0.9599275218914026, 'Germany': 0.19042883179148393, 'Japan': 0.1887361706056662, 'China': 0.10129860713793384, 'F
rance': 0.07078986932066138, 'United Kingdom': 0.05740739709497768, 'South Korea': 0.04911324395823896, 'Canada': 0.032381082242337
995, 'India': 0.016738751284524858}
CSV files saved: Compute_Full_Score.csv + Compute_Target_Score.csv
```

In [7]:

```python
# Canada VS US

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ============================
# 1. Load dataset
# ============================
df = pd.read_csv("3 TOP500Calculation.csv")

# ============================
# 2. Standardize country names
# ============================
df["Country"] = df["Country"].replace({
    "Taiwan": "China",
    "Taiwan, China": "China",
    "Hong Kong": "China",
    "Hong Kong SAR": "China",
    "Hong Kong S.A.R.": "China",
})

# ============================
# 3. Clean numeric columns
# ============================
numeric_cols = [
    "Rmax [TFlop/s]",
    "Total Cores",
    "Accelerator/Co-Processor Cores",
    "Energy Efficiency [GFlops/Watts]"
]

for col in numeric_cols:
```

```python
    df[col] = (
        df[col]
        .astype(str)
        .str.replace(",", "", regex=False)
        .str.extract(r"(\d+\.?\d*)")[0]
        .astype(float)
    )

df = df.fillna(0)

# Add system count
df["SystemCount"] = 1

# ============================
# 4. Filter only Canada & US
# ============================
df_pair = df[df["Country"].isin(["Canada", "United States"])]

# ============================
# 5. Aggregate by Country + Segment
# ============================
segment_stats = df_pair.groupby(["Country", "Segment"]).agg({
    "Rmax [TFlop/s]": "sum",
    "Total Cores": "sum",
    "Accelerator/Co-Processor Cores": "sum",
    "Energy Efficiency [GFlops/Watts]": "mean",
    "SystemCount": "sum"
})

# ============================
# 6. Normalize within each segment
#    Canada vs US share same min/max
# ============================
compute_norm = segment_stats.copy()

for col in ["Rmax [TFlop/s]", "Total Cores",
            "Accelerator/Co-Processor Cores",
            "Energy Efficiency [GFlops/Watts]", "SystemCount"]:

    col_min = segment_stats[col].min()
    col_max = segment_stats[col].max()
    compute_norm[col] = (segment_stats[col] - col_min) / (col_max - col_min + 1e-9)

# Rename normalized columns
compute_norm.rename(columns={
    "Rmax [TFlop/s]": "Rmax_score",
    "Total Cores": "Cores_score",
    "Accelerator/Co-Processor Cores": "GPU_score",
    "Energy Efficiency [GFlops/Watts]": "Energy_score",
    "SystemCount": "Count_score"
}, inplace=True)

# ============================
# 7. Compute weighted final score
# ============================
compute_norm["Compute Score"] = (
    0.50 * compute_norm["Rmax_score"] +
    0.20 * compute_norm["Cores_score"] +
    0.20 * compute_norm["GPU_score"] +
    0.05 * compute_norm["Energy_score"] +
    0.05 * compute_norm["Count_score"]
)

print("\n=== Canada vs US Segment Compute Score ===")
print(compute_norm["Compute Score"])

# ============================
# 8. Plotting
# ============================
compute_plot = compute_norm["Compute Score"].unstack(level=0)  # Segment × Country

compute_plot.plot(kind="bar", figsize=(10,6), edgecolor="black")

plt.title("Compute Score by Segment: Canada vs United States")
plt.ylabel("Compute Score (0–1)")
plt.xticks(rotation=45)
plt.grid(axis="y", alpha=0.3)
plt.legend(title="Country")
plt.show()
```

```
=== Canada vs US Segment Compute Score ===
Country        Segment
Canada         Academic      0.032734
               Government    0.016716
               Industry      0.000038
               Others        0.002388
               Research      0.053907
United States  Academic      0.102037
               Government    0.032793
               Industry      0.243765
               Others        0.006577
               Research      0.971024
               Vendor        0.096703
Name: Compute Score, dtype: float64
```



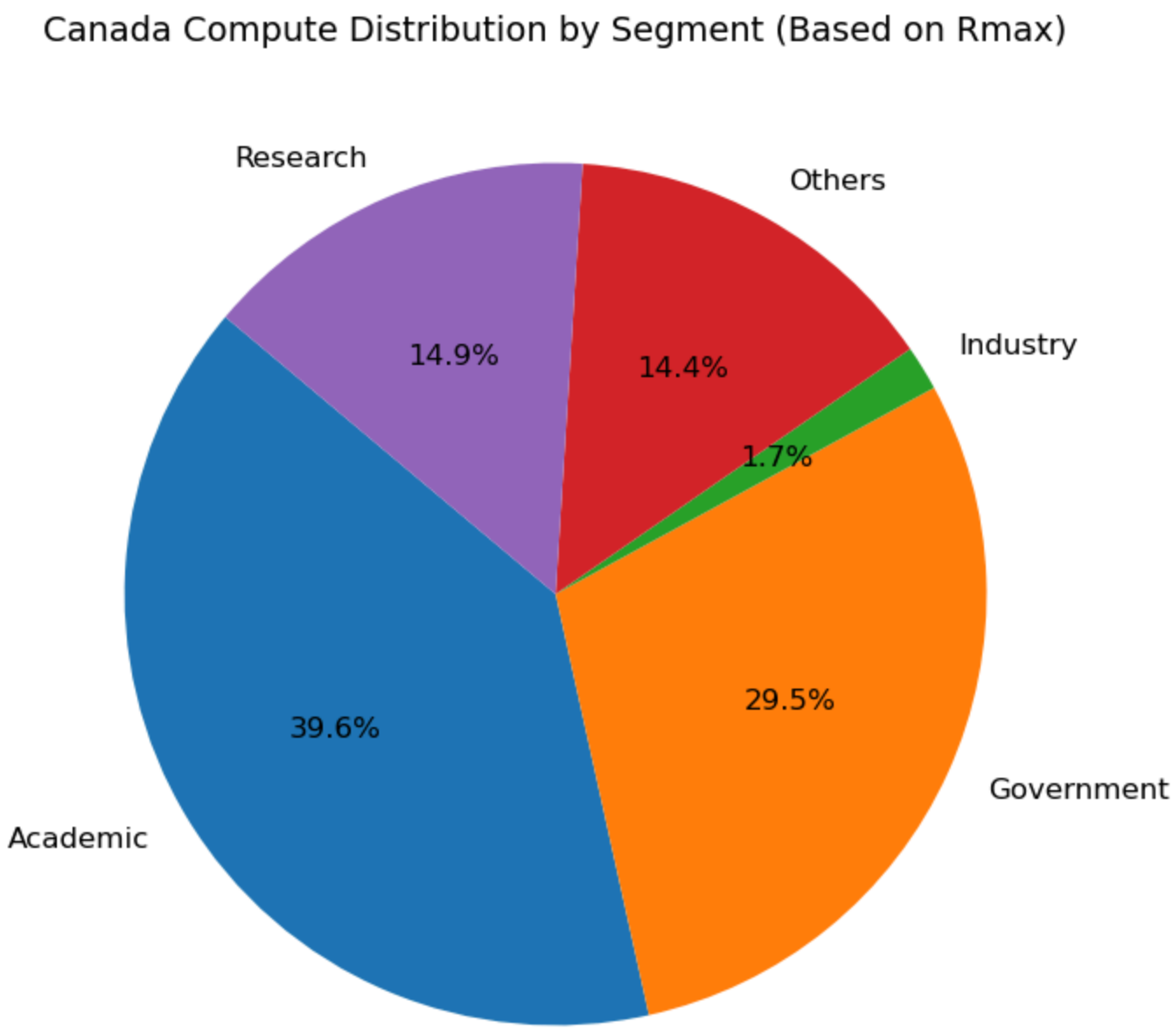
China's TOP500 supercomputer data after 2021 is incomplete because Chinese supercomputing centers largely stopped submitting system performance results to the international TOP500 list for geopolitical and strategic reasons. As a result, China's actual national compute capacity is significantly higher than what is reflected in the publicly available TOP500 dataset.

In [8]:

```
# Compute in Canada


import pandas as pd
import matplotlib.pyplot as plt

# ============================
# Load dataset
# ============================
df = pd.read_csv("3 TOP500Calculation.csv")

# ============================
# Standardize country names
# ============================
df["Country"] = df["Country"].replace({
    "Taiwan": "China",
    "Taiwan, China": "China",
    "Hong Kong": "China",
    "Hong Kong SAR": "China",
    "Hong Kong S.A.R.": "China",
})

# ============================
# Clean numeric Rmax column
# ============================
df["Rmax [TFlop/s]"] = (
    df["Rmax [TFlop/s]"]
    .astype(str)
    .str.replace(",", "", regex=False)
    .str.extract(r"(\d+\.?\d*)")[0]
    .astype(float)
)

df = df.fillna(0)

# ============================
# Filter Canada systems
# ============================
df_canada = df[df["Country"] == "Canada"]

if df_canada.empty:
    print("No Canada entries found in dataset.")
else:
    # Aggregate total compute by Segment
    segment_power = df_canada.groupby("Segment")["Rmax [TFlop/s]"].sum()

    print("\n=== Canada Compute Distribution by Segment ===")
    print(segment_power)

    # ============================
    # Pie Chart
    # ============================
    plt.figure(figsize=(8, 8))
    plt.pie(
```

```
        segment_power,
        labels=segment_power.index,
        autopct="%1.1f%%",
        startangle=140,
        textprops={'fontsize': 12}
    )
    plt.title("Canada Compute Distribution by Segment (Based on Rmax)", fontsize=14)
    plt.show()
```

```
=== Canada Compute Distribution by Segment ===
Segment
Academic      62632.48
Government    46622.56
Industry       2661.56
Others        22740.00
Research      23559.66
Name: Rmax [TFlop/s], dtype: float64
```



4 Industry

In [9]:
```
df_VC_inv = pd.read_csv("4VC investments in AI by country.csv")
df_VC_inv.head()
```

Out[9]:

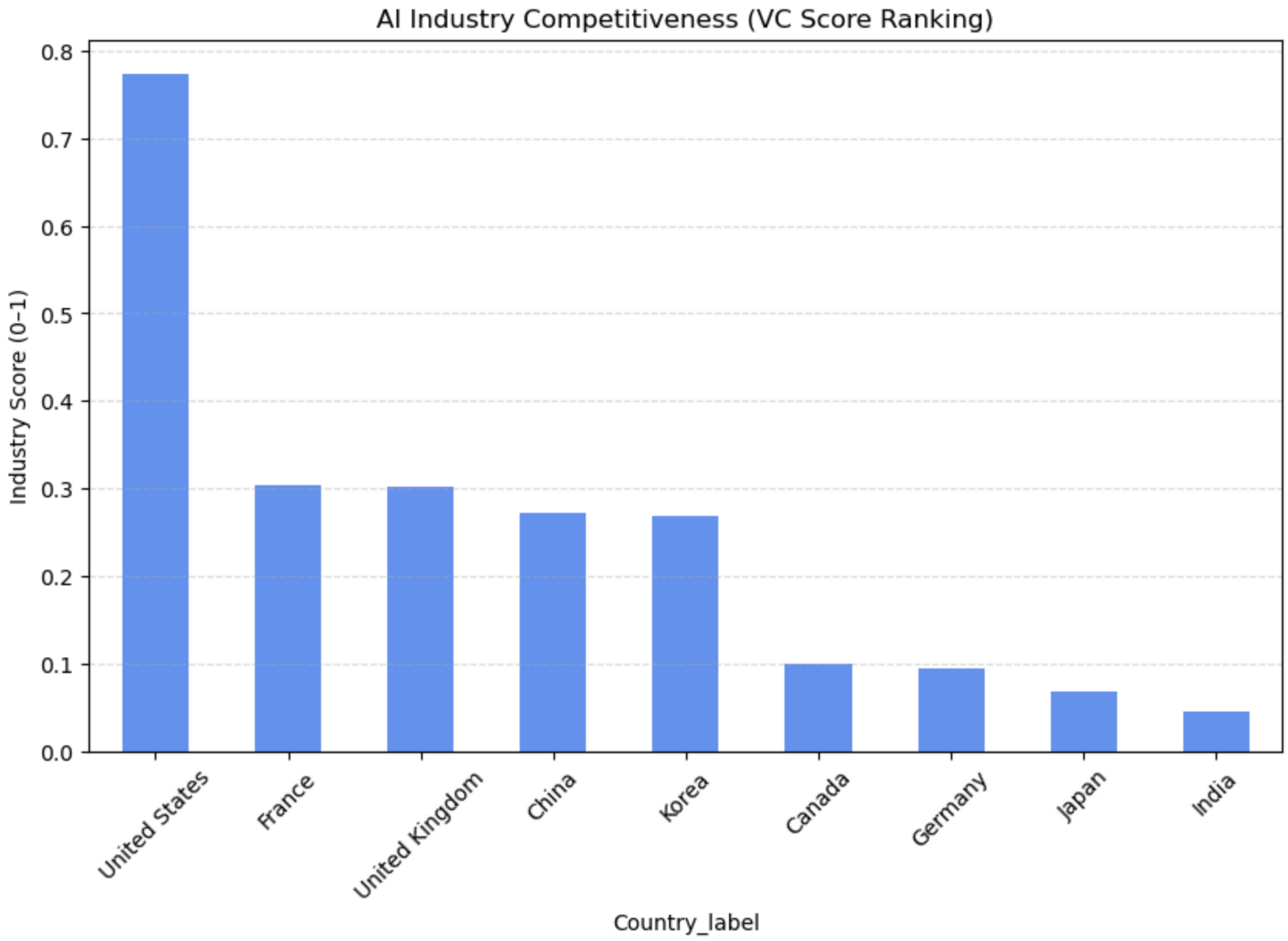
	Country	Country_label	INDUSTRY	STAGE	Sum_of_deals	Year
0	USA	United States	-- All industries --	VC	4872.944386	2012
1	USA	United States	-- All industries --	VC	7320.064116	2013
2	USA	United States	-- All industries --	VC	19395.021055	2014
3	USA	United States	-- All industries --	VC	24895.455316	2015
4	USA	United States	-- All industries --	VC	19395.153007	2016

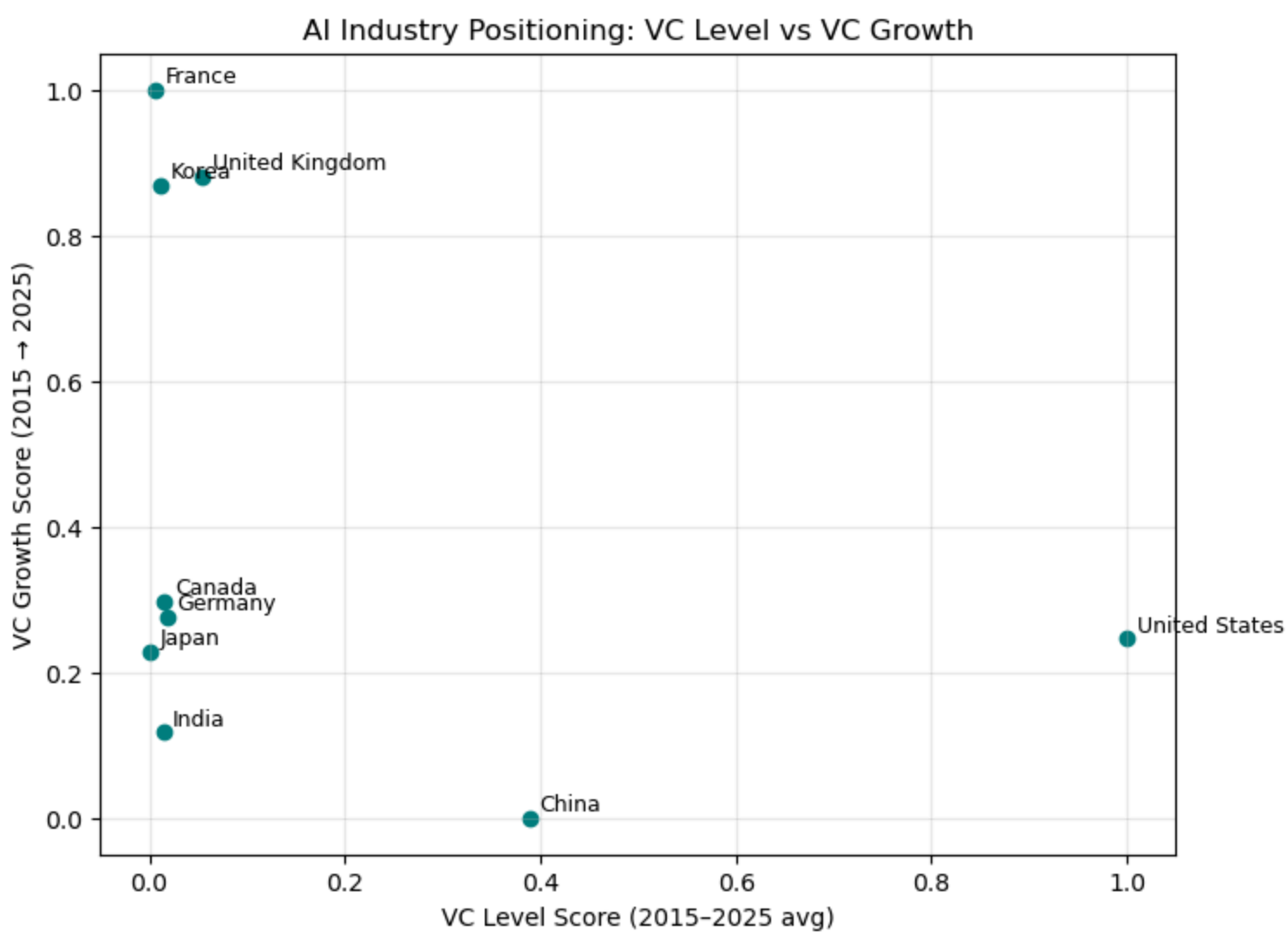
In [10]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ==========================================
# 1. Load dataset
# ==========================================

df_VC_inv = pd.read_csv("4VC investments in AI by country.csv")
print("=== Raw VC Data ===")
print(df_VC_inv.head())

# ==========================================
# 2. Clean + normalize country names
# ==========================================

country_map = {
    "China (People's Republic of)": "China",
    "United States of America": "United States",
    "Republic of Korea": "Korea",
```

```python
    "Korea, Rep.": "Korea",
    "UK": "United Kingdom",
    "U.K.": "United Kingdom",
    "Hong Kong": "China",
    "Taiwan": "China",
    "Japan": "Japan"
}

df_VC_inv["Country_label"] = df_VC_inv["Country_label"].replace(country_map)

target_countries = [
    "Canada", "United States", "China",
    "Germany", "Korea", "United Kingdom",
    "France", "India", "Japan"
]

# Filter only target countries
df_VC_inv = df_VC_inv[df_VC_inv["Country_label"].isin(target_countries)]

# ==========================================
# 3. Pivot: rows = Year, columns = Country
# ==========================================

df_vc = df_VC_inv.groupby(["Country_label", "Year"])["Sum_of_deals"].sum().reset_index()

df_vc_pivot = df_vc.pivot(
    index="Year",
    columns="Country_label",
    values="Sum_of_deals"
).sort_index()

print("\n=== Pivoted Table ===")
print(df_vc_pivot.head())


# ==========================================
# 4. Compute VC Level (2015–2025 mean)
# ==========================================

years_level = [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025]

df_vc_recent = df_vc_pivot.loc[df_vc_pivot.index.isin(years_level)]

vc_level_raw = df_vc_recent.mean()
vc_level_score = (vc_level_raw - vc_level_raw.min()) / (vc_level_raw.max() - vc_level_raw.min())


# ==========================================
# 5. Compute VC Growth (2015 → 2025)
# ==========================================

start_year = 2015
end_year = 2025

vc_growth_raw = (df_vc_pivot.loc[end_year] - df_vc_pivot.loc[start_year]) / df_vc_pivot.loc[start_year]
vc_growth_score = (vc_growth_raw - vc_growth_raw.min()) / (vc_growth_raw.max() - vc_growth_raw.min())


# ==========================================
# 6. Industry Score (weight: Level 0.7 + Growth 0.3)
# ==========================================

industry_score = 0.7 * vc_level_score + 0.3 * vc_growth_score

industry_df = pd.DataFrame({
    "VC Level Score": vc_level_score,
    "VC Growth Score": vc_growth_score,
    "Industry Score": industry_score
}).sort_values("Industry Score", ascending=False)

print("\n=== Final Industry Score ===")
print(industry_df)


# ==========================================
# 7. Bar chart
# ==========================================

plt.figure(figsize=(10, 6))
industry_df["Industry Score"].plot(kind="bar", color="cornflowerblue")
plt.title("AI Industry Competitiveness (VC Score Ranking)")
plt.ylabel("Industry Score (0–1)")
plt.xticks(rotation=45)
plt.grid(axis="y", linestyle="--", alpha=0.4)
plt.show()


# ==========================================
# 8. Scatter plot
# ==========================================

plt.figure(figsize=(8, 6))
plt.scatter(vc_level_score, vc_growth_score, color="teal")

# Add labels
for country in industry_df.index:
```

```python
    plt.text(
        vc_level_score[country] + 0.01,
        vc_growth_score[country] + 0.01,
        country,
        fontsize=9
    )

plt.xlabel("VC Level Score (2015–2025 avg)")
plt.ylabel("VC Growth Score (2015 → 2025)")
plt.title("AI Industry Positioning: VC Level vs VC Growth")
plt.grid(alpha=0.3)
plt.show()


# ============================================
# 9. Export Industry Scores as Dictionary
# ============================================

df_industry = industry_df["Industry Score"].loc[target_countries].to_dict()

print("\n=== Industry Score Dictionary (Target Countries) ===")
print(df_industry)


# ============================================
# Export Industry Score (target only, sorted)
# ============================================


industry_sorted = industry_df.sort_values("Industry Score", ascending=False)


target_sorted = industry_sorted[industry_sorted.index.isin(target_countries)]

target_sorted.to_csv("Industry_Target_Score.csv", encoding="utf-8-sig")

print(target_sorted)
print("Saved: Industry_Target_Score.csv (sorted correctly)")
```

```
=== Raw VC Data ===
  Country  Country_label              INDUSTRY STAGE  Sum_of_deals  Year
0     USA  United States  -- All industries --    VC   4872.944386  2012
1     USA  United States  -- All industries --    VC   7320.064116  2013
2     USA  United States  -- All industries --    VC  19395.021055  2014
3     USA  United States  -- All industries --    VC  24895.455316  2015
4     USA  United States  -- All industries --    VC  19395.153007  2016

=== Pivoted Table ===
Country_label      Canada         China      France     Germany       India  \
Year
2012           136.544316    206.973330  118.039082   24.584660  166.898871
2013           179.762608    559.657997   63.207400  127.745227  116.013629
2014           259.926790   3082.727831   94.581053  180.201204  124.883864
2015           389.917798  10830.048698  198.823085  541.255331  595.403031
2016           738.337715  19226.876453  238.589805  289.435028  394.389139

Country_label       Japan       Korea  United Kingdom  United States
Year
2012            21.428359   97.682777      187.539865    4872.944386
2013            64.137557   16.006547      202.627720    7320.064116
2014           114.172923  106.696504      295.466426   19395.021055
2015           221.706027  111.587725      809.450900   24895.455316
2016           298.620289  174.618527      892.791598   19395.153007

=== Final Industry Score ===
                VC Level Score  VC Growth Score  Industry Score
Country_label
United States         1.000000         0.246264        0.773879
France                0.006315         1.000000        0.304421
United Kingdom        0.053944         0.880153        0.301807
China                 0.388944         0.000000        0.272261
Korea                 0.010775         0.868769        0.268173
Canada                0.015648         0.298117        0.100389
Germany               0.017888         0.275977        0.095315
Japan                 0.000000         0.227959        0.068388
India                 0.014161         0.118086        0.045339
```





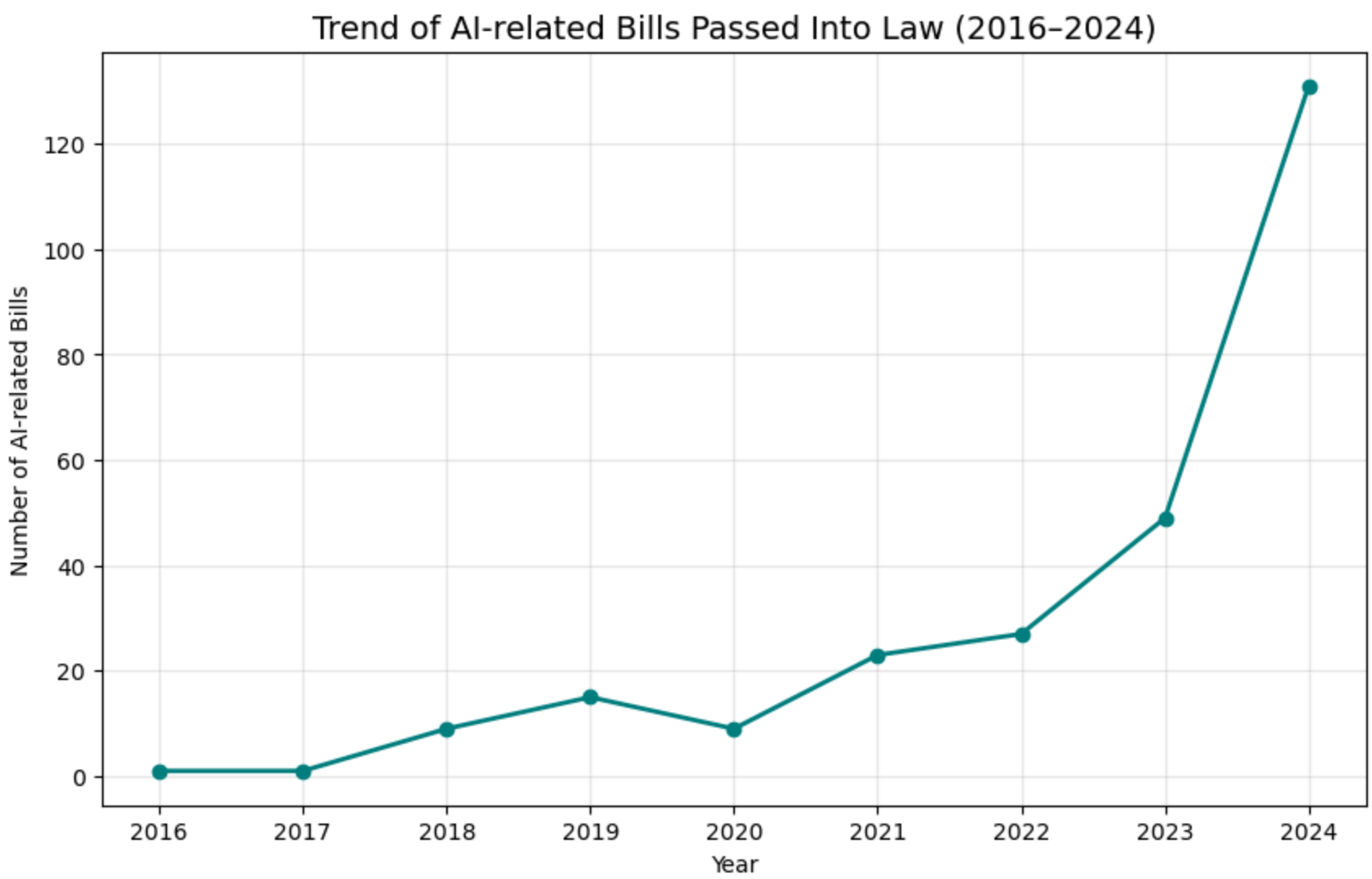

```
=== Industry Score Dictionary (Target Countries) ===
{'Canada': 0.10038870828100092, 'United States': 0.7738793395518463, 'China': 0.27226076662749255, 'Germany': 0.09531471620609908,
'Korea': 0.26817339251193945, 'United Kingdom': 0.3018065195482506, 'France': 0.30442055725406575, 'India': 0.04533880143392236, 'J
apan': 0.06838768348491564}
                VC Level Score  VC Growth Score  Industry Score
Country_label
United States         1.000000         0.246264        0.773879
France                0.006315         1.000000        0.304421
United Kingdom        0.053944         0.880153        0.301807
China                 0.388944         0.000000        0.272261
Korea                 0.010775         0.868769        0.268173
Canada                0.015648         0.298117        0.100389
Germany               0.017888         0.275977        0.095315
Japan                 0.000000         0.227959        0.068388
India                 0.014161         0.118086        0.045339
Saved: Industry_Target_Score.csv (sorted correctly)
```

5 Policy

In [11]:

```
# General outlook

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
df_General_AI_trend = pd.read_csv("5 general AI bill trend.csv")

plt.figure(figsize=(10, 6))
plt.plot(df_General_AI_trend["Year"], df_General_AI_trend["Number of AI-related bills passed into law"],
         marker="o", linewidth=2, color="teal")

plt.title("Trend of AI-related Bills Passed Into Law (2016–2024)", fontsize=14)
plt.xlabel("Year")
plt.ylabel("Number of AI-related Bills")
plt.grid(alpha=0.3)

plt.xticks(df_General_AI_trend["Year"])
plt.show()
```

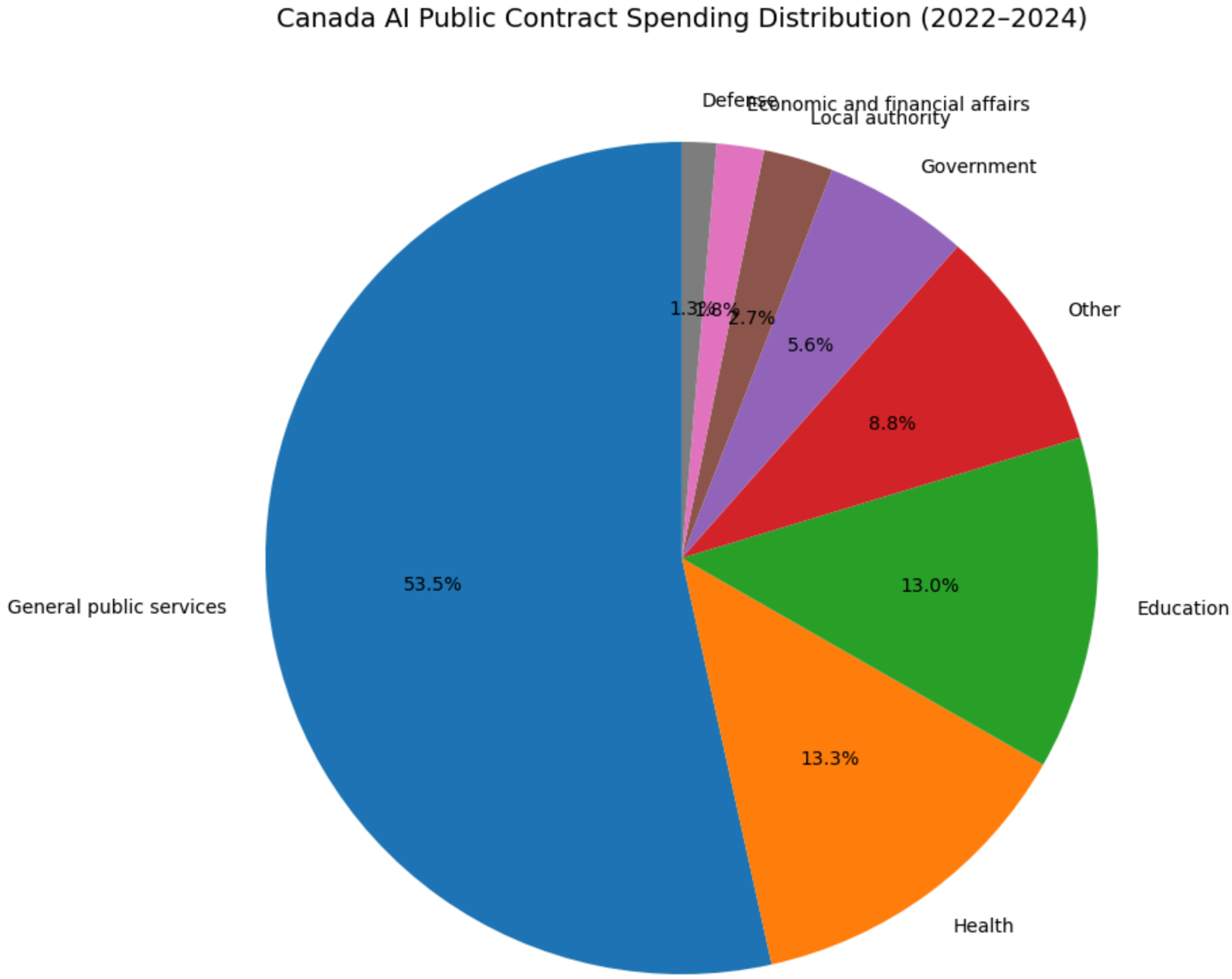


In [12]:

```
# AI in Canada (outlook)

import pandas as pd
import matplotlib.pyplot as plt

# ============================
# 1. Load contracts data
# ============================

df_con = pd.read_csv("5 AI related contracts %.csv")

print("=== Raw Contracts Data ===")
print(df_con.head())


# ============================
# 2. Filter for last 3 years (2022–2024)
# ============================

df_recent = df_con[df_con["Year"].isin([2022, 2023, 2024])]


# ============================
# 3. Aggregate by sector (Label)
#    Use mean spending per sector
# ============================

sector_spending = (
    df_recent.groupby("Label")["Public spending on AI-related contracts (% of total)"]
    .mean()
    .sort_values(ascending=False)
)

print("\n=== Canada AI Contract Spending (2022–2024 avg, by sector) ===")
print(sector_spending)


# ============================
# 4. Plot Pie Chart
# ============================

plt.figure(figsize=(10, 10))

plt.pie(
    sector_spending,
    labels=sector_spending.index,
    autopct='%1.1f%%',
    startangle=90,
```

```
    textprops={'fontsize': 10}
)

plt.title("Canada AI Public Contract Spending Distribution (2022–2024)", fontsize=14)
plt.show()
```

```
=== Raw Contracts Data ===
   Year  Public spending on AI-related contracts (% of total)    Label
0  2013                                             0.000000     Defense
1  2014                                             0.000000     Defense
2  2015                                             0.000000     Defense
3  2016                                             0.000000     Defense
4  2017                                             0.009886     Defense

=== Canada AI Contract Spending (2022–2024 avg, by sector) ===
Label
General public services           0.534654
Health                           0.132652
Education                        0.129501
Other                            0.087896
Government                       0.056457
Local authority                  0.027087
Economic and financial affairs   0.018389
Defense                          0.013364
Name: Public spending on AI-related contracts (% of total), dtype: float64
```

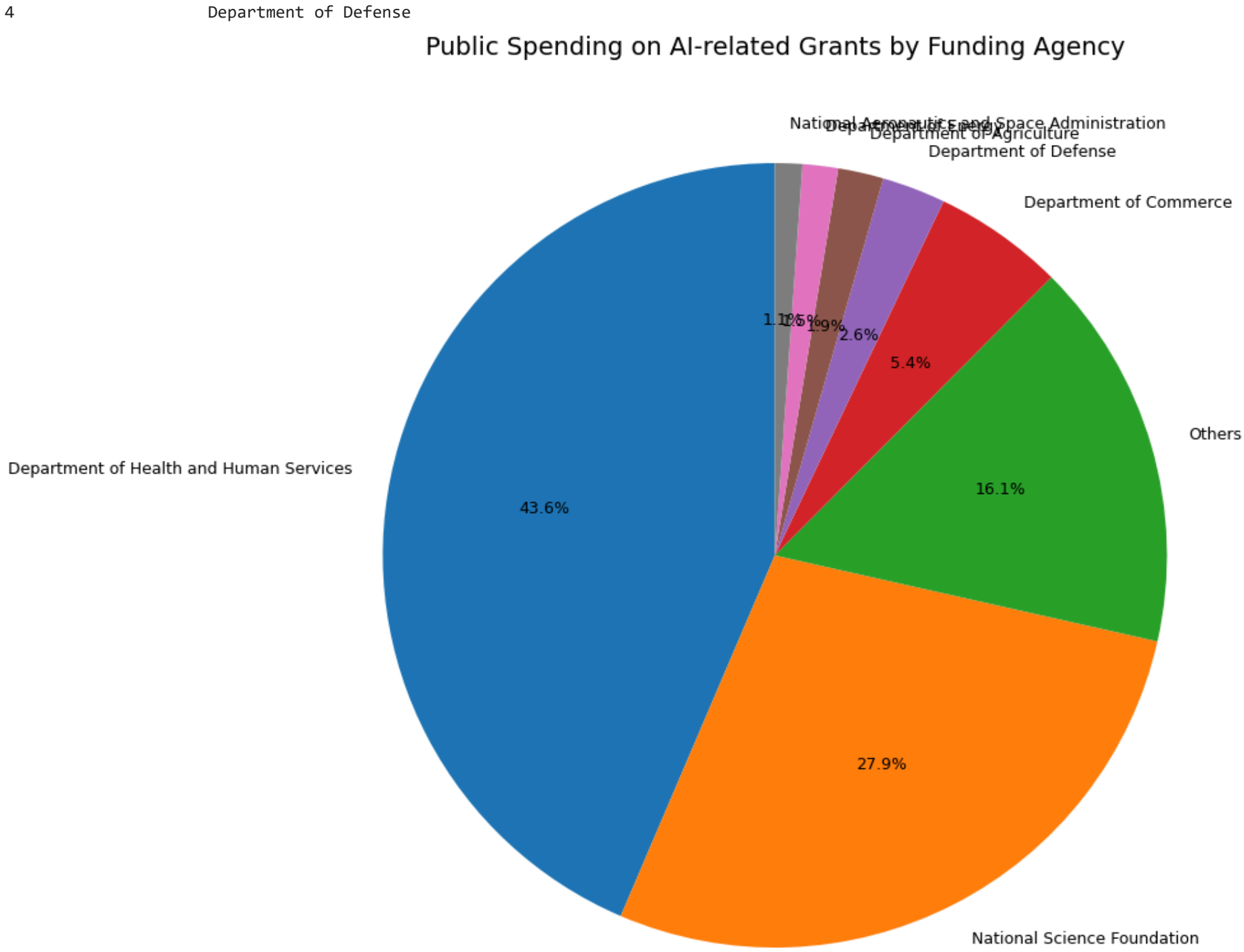


In [13]:

```
#AI in Canada (Outlook)

import pandas as pd
import matplotlib.pyplot as plt

# =======================
# 1. Load data
# =======================

df_grants = pd.read_csv("5 Public spending on AI %.csv")

print("=== Raw Grants Data ===")
print(df_grants.head())


# =======================
# 2. Clean percentage column
# =======================

# Remove "%" and convert to float
df_grants["Public spending on AI-related grants (% of total)"] = (
    df_grants["Public spending on AI-related grants (% of total)"]
```

```
    .str.replace('%', '', regex=False)
    .astype(float)
)

# Extract values
values = df_grants["Public spending on AI-related grants (% of total)"]
labels = df_grants["Funding agency"]


# =======================
# 3. Plot pie chart
# =======================

plt.figure(figsize=(10, 10))

plt.pie(
    values,
    labels=labels,
    autopct='%1.1f%%',
    startangle=90,
    textprops={'fontsize': 9}
)

plt.title("Public Spending on AI-related Grants by Funding Agency", fontsize=14)
plt.show()
```

```
=== Raw Grants Data ===
  Public spending on AI-related grants (% of total)  \
0                                            43.57%
1                                            27.91%
2                                            16.06%
3                                             5.38%
4                                             2.62%

                          Funding agency
0  Department of Health and Human Services
1              National Science Foundation
2                                   Others
3                    Department of Commerce
4                     Department of Defense
```



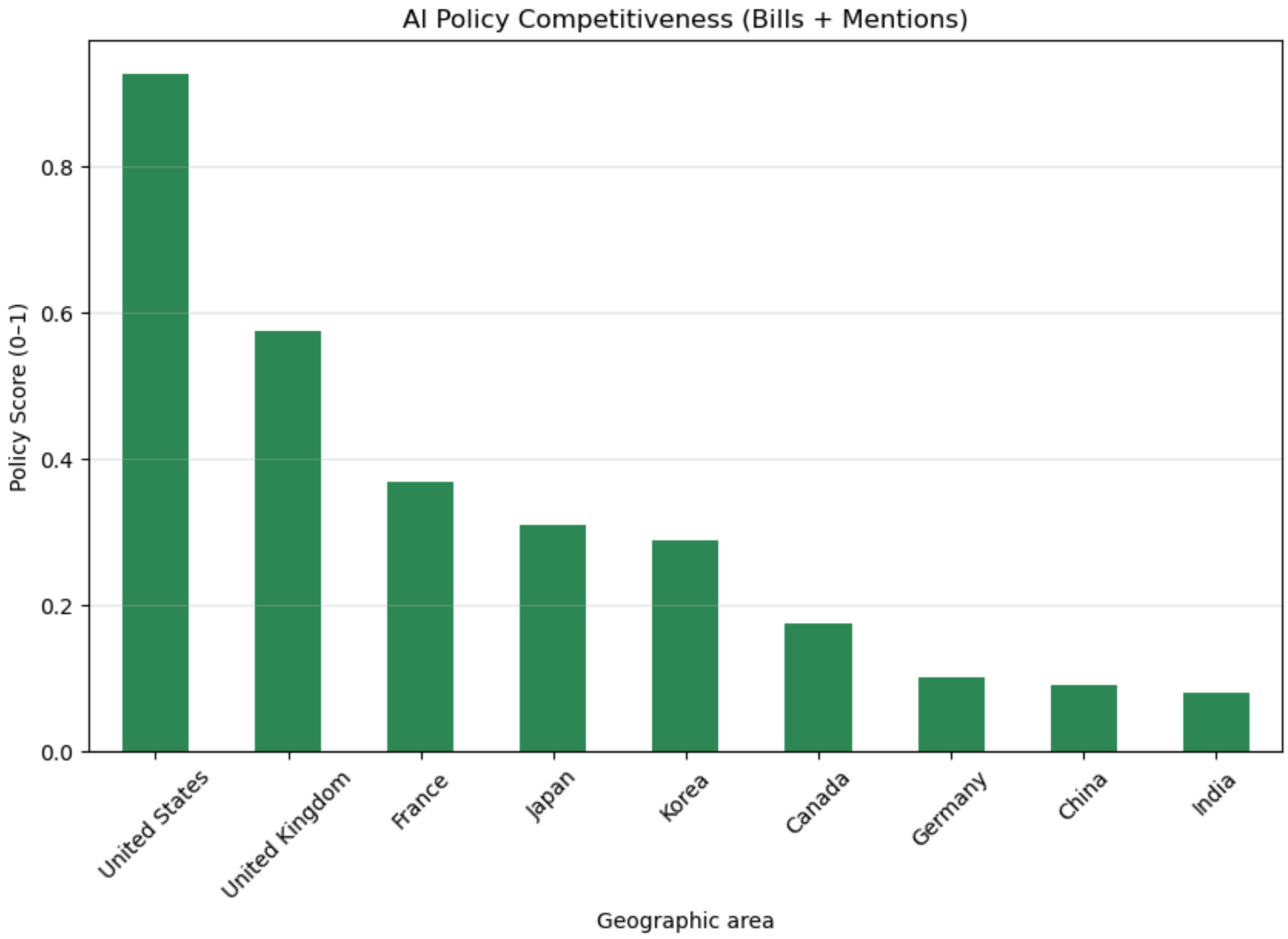
In [14]:

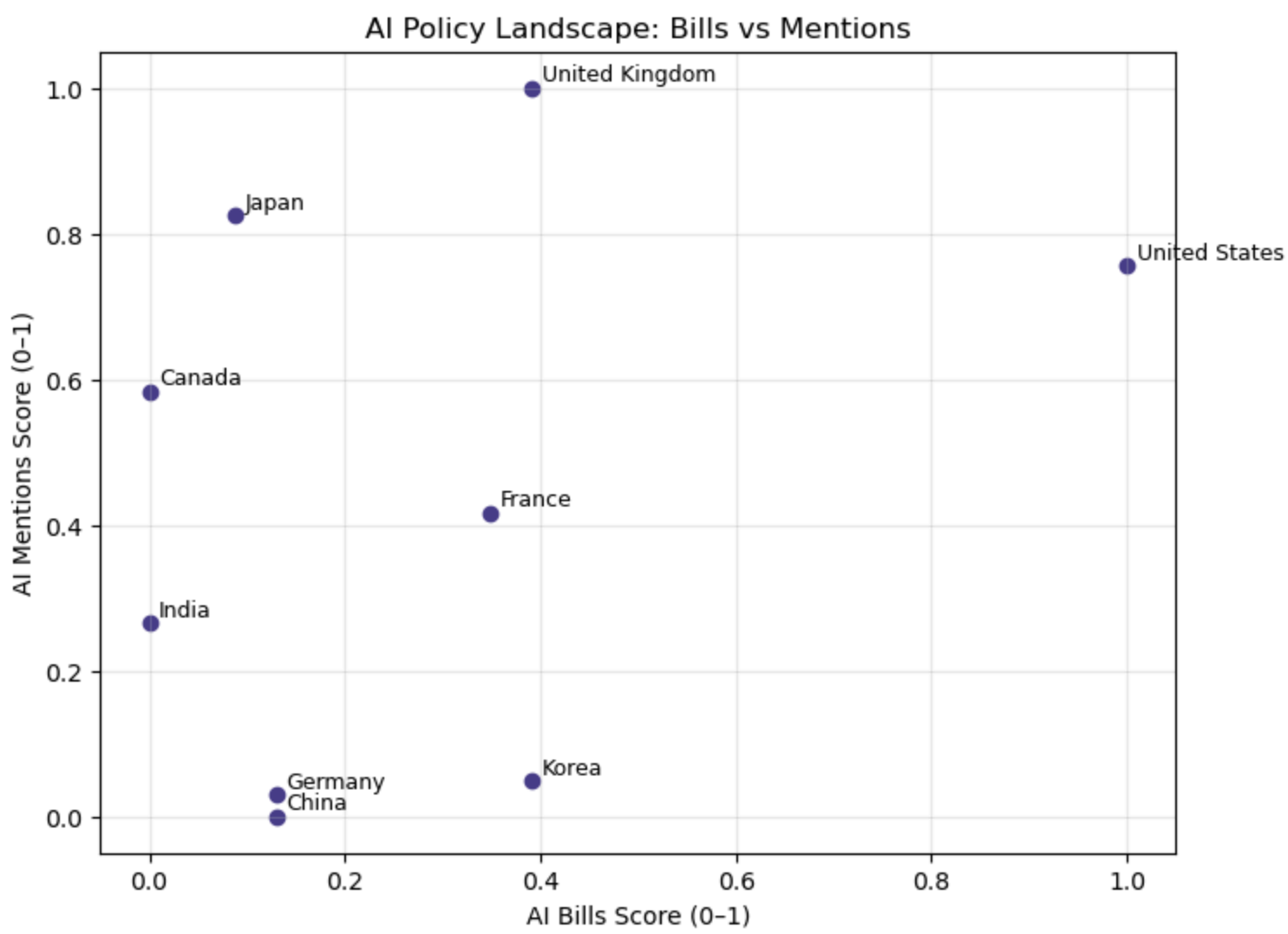
```
# Canada VS World

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ==========================================
# 1. Load Data
# ==========================================
```

```python
df = pd.read_csv("5 AI bill and AI mention.csv")

print("=== Raw Data Preview ===")
print(df.head())
print(df.columns)


# ==============================================
# 2. Standardize Country Names
# ==============================================

country_map = {
    "United States of America": "United States",
    "China (People's Republic of)": "China",
    "South Korea": "Korea",
    "UK": "United Kingdom",
    "U.K.": "United Kingdom",
}

df["Geographic area"] = df["Geographic area"].replace(country_map)


# ==============================================
# 3. Filter the 9 target countries
# ==============================================

target_countries = [
    "Canada", "United States", "China",
    "Germany", "Korea", "United Kingdom",
    "France", "India", "Japan"
]

df = df[df["Geographic area"].isin(target_countries)]


# ==============================================
# 4. Extract metrics (single-year data)
# ==============================================

bills_raw = df.set_index("Geographic area")["Number of AI-related bills passed into law"]
mentions_raw = df.set_index("Geographic area")["Number of AI mentions"]

print("\n=== Bills Raw ===")
print(bills_raw)

print("\n=== Mentions Raw ===")
print(mentions_raw)


# ==============================================
# 5. Min-Max Normalization
# ==============================================

bills_score = (bills_raw - bills_raw.min()) / (bills_raw.max() - bills_raw.min())
mentions_score = (mentions_raw - mentions_raw.min()) / (mentions_raw.max() - mentions_raw.min())


# ==============================================
# 6. Final Policy Score (Bills 0.7 + Mentions 0.3)
# ==============================================

policy_score = 0.7 * bills_score + 0.3 * mentions_score

policy_df = pd.DataFrame({
    "AI Bills Score": bills_score,
    "AI Mentions Score": mentions_score,
    "Policy Score": policy_score
}).sort_values("Policy Score", ascending=False)

print("\n=== Final Policy Score Ranking ===")
print(policy_df)


# ==============================================
# 7. Plot 1 — Policy Score Ranking (Bar Chart)
# ==============================================

plt.figure(figsize=(10,6))
policy_df["Policy Score"].plot(kind="bar", color="seagreen")

plt.title("AI Policy Competitiveness (Bills + Mentions)")
plt.ylabel("Policy Score (0–1)")
plt.xticks(rotation=45)
plt.grid(axis="y", alpha=0.3)
plt.show()


# ==============================================
# 8. Plot 2 — Bills vs Mentions Scatter Plot
# ==============================================

plt.figure(figsize=(8,6))
plt.scatter(bills_score, mentions_score, color="darkslateblue")

for country in policy_df.index:
    plt.text(
```

```python
        bills_score[country] + 0.01,
        mentions_score[country] + 0.01,
        country,
        fontsize=9
    )

plt.xlabel("AI Bills Score (0–1)")
plt.ylabel("AI Mentions Score (0–1)")
plt.title("AI Policy Landscape: Bills vs Mentions")
plt.grid(alpha=0.3)
plt.show()



# ==========================================
# 9. Export Policy Score Dictionary
# ==========================================

df_policy = policy_df["Policy Score"].loc[target_countries].to_dict()

print("\n=== Policy Score Dictionary (Target Countries) ===")
print(df_policy)


# ==================================================
# 10. Export Policy Scores to CSV (sorted descending)
# ==================================================

policy_sorted = policy_df.sort_values("Policy Score", ascending=False)


policy_target_sorted = policy_sorted[policy_sorted.index.isin(target_countries)]



policy_target_sorted.to_csv("Policy_Target_Score.csv", encoding="utf-8-sig")

print("CSV saved: Policy_Target_Score.csv (sorted descending)")
print(policy_target_sorted)
```

```
=== Raw Data Preview ===
  Geographic area  Number of AI-related bills passed into law  \
0   United States                                           24
1        Portugal                                           20
2          Russia                                           20
3         Belgium                                           18
4           Spain                                           12

   Number of AI mentions
0                    538
1                     41
2                    108
3                     18
4                   1200
Index(['Geographic area', 'Number of AI-related bills passed into law',
       'Number of AI mentions'],
      dtype='object')

=== Bills Raw ===
Geographic area
United States     24
Korea             10
United Kingdom    10
France             9
China              4
Germany            4
Japan              3
Canada             1
India              1
Name: Number of AI-related bills passed into law, dtype: int64

=== Mentions Raw ===
Geographic area
United States     538
Korea              40
United Kingdom    710
France            299
China               6
Germany            27
Japan             587
Canada            417
India             193
Name: Number of AI mentions, dtype: int64

=== Final Policy Score Ranking ===
                 AI Bills Score  AI Mentions Score  Policy Score
Geographic area
United States          1.000000           0.755682      0.926705
United Kingdom         0.391304           1.000000      0.573913
France                 0.347826           0.416193      0.368336
Japan                  0.086957           0.825284      0.308455
Korea                  0.391304           0.048295      0.288402
Canada                 0.000000           0.583807      0.175142
Germany                0.130435           0.029830      0.100253
China                  0.130435           0.000000      0.091304
India                  0.000000           0.265625      0.079687
```






```
=== Policy Score Dictionary (Target Countries) ===
{'Canada': 0.17514204545454545, 'United States': 0.9267045454545454, 'China': 0.09130434782608694, 'Germany': 0.10025321146245059,
'Korea': 0.2884016798418972, 'United Kingdom': 0.5739130434782609, 'France': 0.3683362154150197, 'India': 0.0796875, 'Japan': 0.308
4547924901186}
CSV saved: Policy_Target_Score.csv (sorted descending)
                 AI Bills Score  AI Mentions Score  Policy Score
Geographic area
United States          1.000000           0.755682      0.926705
United Kingdom         0.391304           1.000000      0.573913
France                 0.347826           0.416193      0.368336
Japan                  0.086957           0.825284      0.308455
Korea                  0.391304           0.048295      0.288402
Canada                 0.000000           0.583807      0.175142
Germany                0.130435           0.029830      0.100253
China                  0.130435           0.000000      0.091304
India                  0.000000           0.265625      0.079687
```

6 Result

In [15]:
```python
def replace_korea(d):
    new_dict = {}
    for k, v in d.items():
        if k == "Korea":
            new_dict["South Korea"] = v
        else:
```

```
            new_dict[k] = v
    return new_dict

df_research = replace_korea(df_research)
df_talent = talent_target_sorted["Talent Score"].to_dict()
df_talent = replace_korea(df_talent)

df_compute = replace_korea(df_compute)
df_industry = replace_korea(df_industry)
df_policy = replace_korea(df_policy)

# Print results
print("\n=== Research Scores ===")
print(df_research)

print("\n=== Talent Scores ===")
print(df_talent)

print("\n=== Compute Scores ===")
print(df_compute)

print("\n=== Industry Scores ===")
print(df_industry)

print("\n=== Policy Scores ===")
print(df_policy)
```

```
=== Research Scores ===
{'United States': 0.7712519660242639, 'China': 0.6180195472681431, 'South Korea': 0.2727663299264079, 'United Kingdom': 0.256310775
8556209, 'Germany': 0.24698535607933575, 'Canada': 0.2300600164719307, 'France': 0.16403105909300955, 'India': 0.15, 'Japan': 0.094
83143228122126}

=== Talent Scores ===
{'United States': 1.0, 'China': 0.194811313393087, 'India': 0.18352029795737082, 'United Kingdom': 0.18025369979013497, 'Canada':
0.14903627091126095, 'Germany': 0.1318836571672481, 'France': 0.11573076968534175, 'South Korea': 0.07776589836598571, 'Japan': 0.0
3794351562103028}

=== Compute Scores ===
{'United States': 0.9599275218914026, 'Germany': 0.19042883179148393, 'Japan': 0.1887361706056662, 'China': 0.10129860713793384, 'F
rance': 0.07078986932066138, 'United Kingdom': 0.05740739709497768, 'South Korea': 0.04911324395823896, 'Canada': 0.032381082242337
995, 'India': 0.016738751284524858}

=== Industry Scores ===
{'Canada': 0.10038870828100092, 'United States': 0.7738793395518463, 'China': 0.27226076662749255, 'Germany': 0.09531471620609908,
'South Korea': 0.26817339251193945, 'United Kingdom': 0.3018065195482506, 'France': 0.30442055725406575, 'India': 0.045338801433922
36, 'Japan': 0.06838768348491564}

=== Policy Scores ===
{'Canada': 0.17514204545454545, 'United States': 0.9267045454545454, 'China': 0.09130434782608694, 'Germany': 0.10025321146245059,
'South Korea': 0.2884016798418972, 'United Kingdom': 0.5739130434782609, 'France': 0.3683362154150197, 'India': 0.0796875, 'Japan':
0.3084547924901186}
```

In [16]:

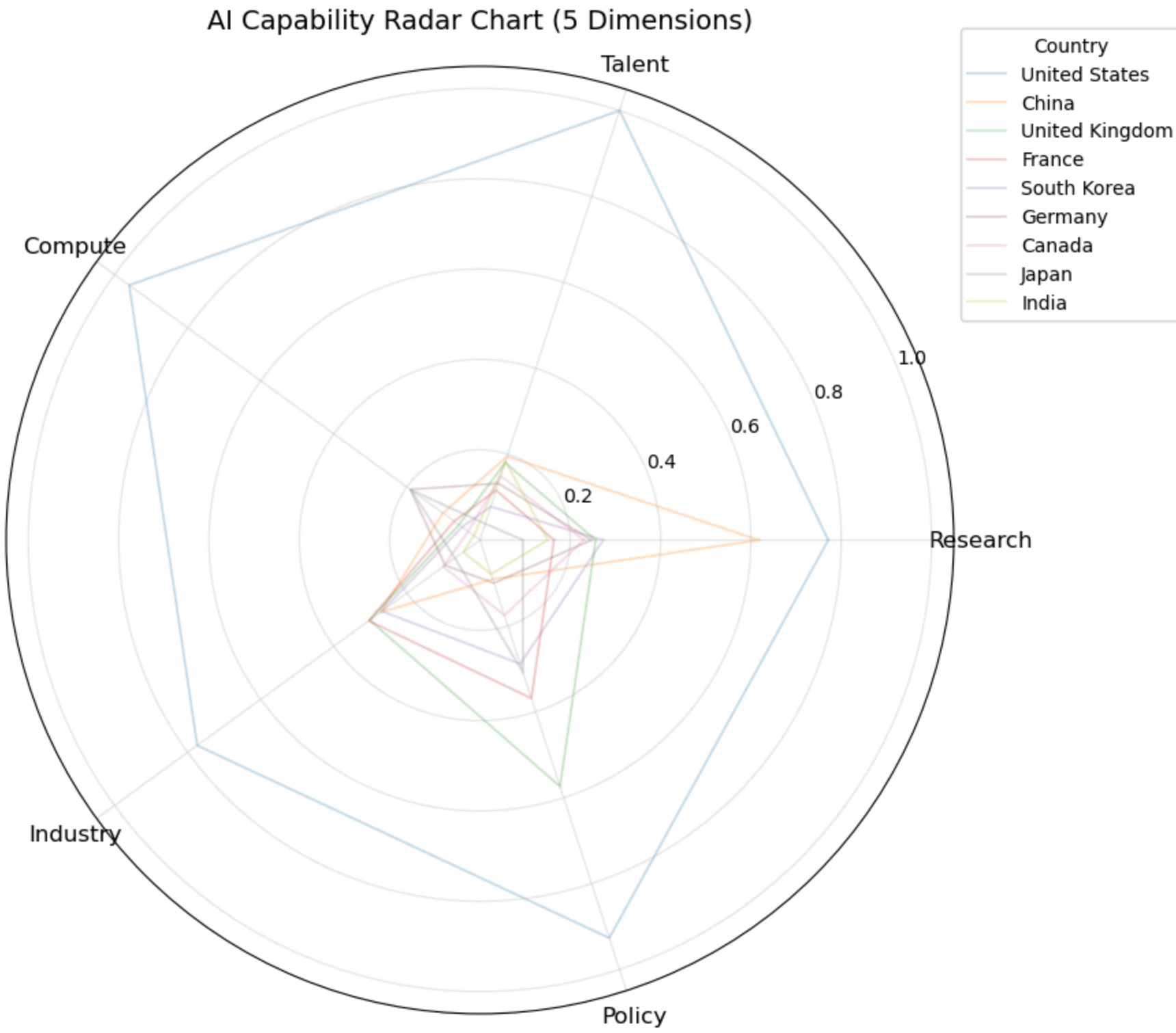
```
import pandas as pd


all_countries = list(set(
    list(df_research.keys()) +
    list(df_talent.keys()) +
    list(df_compute.keys()) +
    list(df_industry.keys()) +
    list(df_policy.keys())
))

# ==============================================
# Combine into 1 dataframe
# ==============================================

df = pd.DataFrame({
    "Research": pd.Series(df_research),
    "Talent": pd.Series(df_talent),
    "Compute": pd.Series(df_compute),
    "Industry": pd.Series(df_industry),
    "Policy": pd.Series(df_policy)
}).reindex(all_countries).fillna(0)

# ==============================================
# weight
# ==============================================

weights = {
    "Research":0.20,
    "Talent":0.25,
    "Compute":0.25,
    "Industry":0.15,
    "Policy":0.15
}

# ==============================================
# Calculate final score
# ==============================================

df["Total_Score"] = (
    df["Research"] * weights["Research"] +
    df["Talent"] * weights["Talent"] +
    df["Compute"] * weights["Compute"] +
    df["Industry"] * weights["Industry"] +
```

```
    df["Policy"] * weights["Policy"]
)


df = df.sort_values("Total_Score", ascending=False)

print("\n=== Final Weighted Total AI Capability Score ===")
print(df)


# ==================================================
# Export Final Total AI Capability Score
# ==================================================


df_sorted = df.sort_values("Total_Score", ascending=False)

df_target_sorted = df_sorted[df_sorted.index.isin(target)]



df_target_sorted.to_csv("Total_AI_Target_Score.csv", encoding="utf-8-sig")

# 4. 可选：字典
df_total_dict = df["Total_Score"].to_dict()

print("Files saved:")
print(" - Total_AI_Target_Score.csv (sorted)")
print(df_target_sorted)
```

```
=== Final Weighted Total AI Capability Score ===
               Research    Talent   Compute  Industry    Policy  Total_Score
United States   0.771252  1.000000  0.959928  0.773879  0.926705     0.899320
China           0.618020  0.194811  0.101299  0.272261  0.091304     0.252166
United Kingdom  0.256311  0.180254  0.057407  0.301807  0.573913     0.242035
France          0.164031  0.115731  0.070790  0.304421  0.368336     0.180350
South Korea     0.272766  0.077766  0.049113  0.268173  0.288402     0.169759
Germany         0.246985  0.131884  0.190429  0.095315  0.100253     0.159310
Canada          0.230060  0.149036  0.032381  0.100389  0.175142     0.132696
Japan           0.094831  0.037944  0.188736  0.068388  0.308455     0.132163
India           0.150000  0.183520  0.016739  0.045339  0.079687     0.098819
Files saved:
 - Total_AI_Target_Score.csv (sorted)
               Research    Talent   Compute  Industry    Policy  Total_Score
United States   0.771252  1.000000  0.959928  0.773879  0.926705     0.899320
China           0.618020  0.194811  0.101299  0.272261  0.091304     0.252166
United Kingdom  0.256311  0.180254  0.057407  0.301807  0.573913     0.242035
France          0.164031  0.115731  0.070790  0.304421  0.368336     0.180350
South Korea     0.272766  0.077766  0.049113  0.268173  0.288402     0.169759
Germany         0.246985  0.131884  0.190429  0.095315  0.100253     0.159310
Canada          0.230060  0.149036  0.032381  0.100389  0.175142     0.132696
Japan           0.094831  0.037944  0.188736  0.068388  0.308455     0.132163
India           0.150000  0.183520  0.016739  0.045339  0.079687     0.098819
```

In [17]:

```
import matplotlib.pyplot as plt
from math import pi


# df = df.set_index("Country")

# ================================
# 5 dimension of Radar
# ================================
metrics = ["Research", "Talent", "Compute", "Industry", "Policy"]
N = len(metrics)

angles = [n / float(N) * 2 * pi for n in range(N)]
angles += angles[:1]

plt.figure(figsize=(9, 9))

for country in df.index:
    values = df.loc[country, metrics].tolist()
    values += values[:1]
    plt.polar(angles, values, linewidth=1.2, alpha=0.25, label=country)

plt.xticks(angles[:-1], metrics, fontsize=12)
plt.title("AI Capability Radar Chart (5 Dimensions)", fontsize=14, pad=20)


plt.legend(bbox_to_anchor=(1.25, 1.05), title="Country")

plt.grid(alpha=0.3)
plt.show()
```



In [18]:

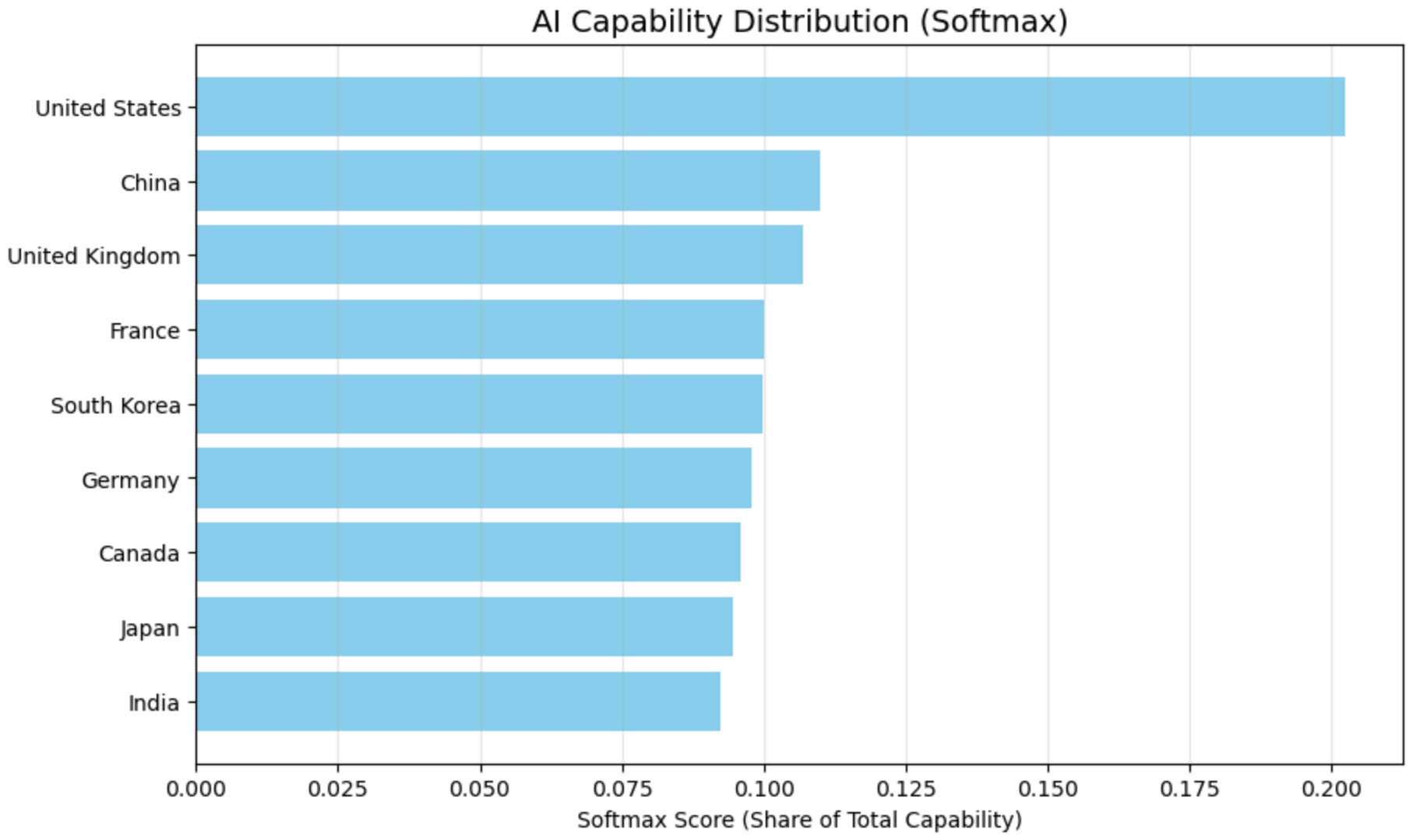
```
import pandas as pd
import numpy as np

# ========================================
# 1. 5 demension of dictionary
# ========================================


# df_research = {...}
# df_talent = {...}
# df_compute = {...}
# df_industry = {...}
# df_policy = {...}

# ========================================
# 2. gather all countries
# ========================================
all_countries = sorted(list(set(
    list(df_research.keys()) +
    list(df_talent.keys()) +
    list(df_compute.keys()) +
    list(df_industry.keys()) +
    list(df_policy.keys())
)))

# ========================================
# 3. DataFrame
# ========================================

df = pd.DataFrame({
    "Country": all_countries,
    "Research": [df_research.get(c, 0) for c in all_countries],
    "Talent":   [df_talent.get(c, 0) for c in all_countries],
    "Compute":  [df_compute.get(c, 0) for c in all_countries],
    "Industry": [df_industry.get(c, 0) for c in all_countries],
    "Policy":   [df_policy.get(c, 0) for c in all_countries],
})

# ========================================
# 4. weight
# ========================================
weights = {
    "Research": 0.25,
    "Talent":   0.25,
    "Compute":  0.20,
    "Industry": 0.15,
    "Policy":   0.15
}
```

```python
# ========================================
# 5.Raw Total Score
# ========================================
df["Raw_Total"] = (
    df["Research"] * weights["Research"] +
    df["Talent"]   * weights["Talent"] +
    df["Compute"]  * weights["Compute"] +
    df["Industry"] * weights["Industry"] +
    df["Policy"]   * weights["Policy"]
)

# ========================================
# 6. Softmax
# ========================================
def softmax(x):
    x = np.array(x)
    x = x - np.max(x)
    exp_x = np.exp(x)
    return exp_x / exp_x.sum()

df["Softmax_Score"] = softmax(df["Raw_Total"])

# ========================================
# 7. result
# ========================================
df = df.sort_values("Softmax_Score", ascending=False)

print("\n=== 🌍 Final AI Capability Ranking (Softmax) ===")
print(df)


# ====================================================
# 8. Export Softmax Results to CSV
# ====================================================

# Full
df.to_csv("Total_AI_Softmax_Full_Score.csv", encoding="utf-8-sig", index=False)

# Target
target = [
    "Canada","United States","China","Germany","South Korea",
    "United Kingdom","France","India","Japan"
]

df.loc[df["Country"].isin(target)].to_csv(
    "Total_AI_Softmax_Target_Score.csv",
    encoding="utf-8-sig",
    index=False
)

print("Files saved:")
print(" - Total_AI_Softmax_Full_Score.csv")
print(" - Total_AI_Softmax_Target_Score.csv")
```

```
=== 🌍 Final AI Capability Ranking (Softmax) ===
          Country  Research    Talent   Compute  Industry    Policy  \
8   United States  0.771252  1.000000  0.959928  0.773879  0.926705
1           China  0.618020  0.194811  0.101299  0.272261  0.091304
7  United Kingdom  0.256311  0.180254  0.057407  0.301807  0.573913
2          France  0.164031  0.115731  0.070790  0.304421  0.368336
6     South Korea  0.272766  0.077766  0.049113  0.268173  0.288402
3         Germany  0.246985  0.131884  0.190429  0.095315  0.100253
0          Canada  0.230060  0.149036  0.032381  0.100389  0.175142
5           Japan  0.094831  0.037944  0.188736  0.068388  0.308455
4           India  0.150000  0.183520  0.016739  0.045339  0.079687

   Raw_Total  Softmax_Score
8   0.889886       0.202568
1   0.278002       0.109858
7   0.251981       0.107036
2   0.185012       0.100103
6   0.180942       0.099696
3   0.162138       0.097839
0   0.142580       0.095944
5   0.127467       0.094505
4   0.105482       0.092450
Files saved:
 - Total_AI_Softmax_Full_Score.csv
 - Total_AI_Softmax_Target_Score.csv
```

In [19]:

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plt.barh(df["Country"], df["Softmax_Score"], color="skyblue")
plt.title("AI Capability Distribution (Softmax)", fontsize=14)
plt.xlabel("Softmax Score (Share of Total Capability)")
plt.gca().invert_yaxis()
plt.grid(axis="x", alpha=0.3)

plt.show()
```


AI Capability Distribution (Softmax)
United States
China
United Kingdom
France
South Korea
Germany
Canada
Japan
India
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
Softmax Score (Share of Total Capability)

In [ ]:

In [ ]:

AI Score Dataset Cleaning, Organization, and Final Merge

MIE1624 Group11

This Jupyter Notebook serves as a second-stage processor that cleans and organizes the CSV files generated by the previous notebook. Its purpose is to ensure all AI domain score data is consistently formatted, merged correctly, and optimized for LLM-friendly reading.

The final output is a unified file (`Combined_AI_Target_Scores.csv`).

Both notebooks will be exported as PDFs and combined into a single document to clearly present the full data-processing methodology for the LLM.

In [1]:
```
import pandas as pd

# Load original CSV files
compute         = pd.read_csv("Compute_Target_Score.csv")
industry        = pd.read_csv("Industry_Target_Score.csv")
policy          = pd.read_csv("Policy_Target_Score.csv")
research        = pd.read_csv("Research_Score.csv")
talent          = pd.read_csv("Talent_Target_Score.csv")
total_raw       = pd.read_csv("Total_AI_Target_Score.csv")
total_softmax   = pd.read_csv("Total_AI_Softmax_Target_Score.csv")
```

In [2]:
```
industry    = industry.rename(columns={"Country_label": "Country"})
policy      = policy.rename(columns={"Geographic area": "Country"})
research    = research.rename(columns={"Country/territory_label": "Country"})
talent      = talent.rename(columns={"Geographic area": "Country"})
total_raw   = total_raw.rename(columns={"Unnamed: 0": "Country"})
```

In [3]:
```
compute.columns
```

Out[3]:
```
Index(['Country', 'Rmax_score', 'Cores_score', 'GPU_score', 'Energy_score',
       'Count_score', 'Compute Score'],
      dtype='object')
```

In [4]:
```
compute.columns = [col.replace("_score", "_Score") for col in compute.columns]
compute.columns = [col.replace(" Score", "_Score") for col in compute.columns]
```

In [5]:
```
industry.columns
```

Out[5]:
```
Index(['Country', 'VC Level Score', 'VC Growth Score', 'Industry Score'], dtype='object')
```

In [6]:
```
industry.columns = industry.columns.str.replace(" ", "_")
```

In [7]:
```
policy.columns = policy.columns.str.replace(" ", "_")
```

In [8]:
```
research.columns = research.columns.str.replace(" ", "_")
```

In [9]:
```
talent.columns
```

Out[9]:
```
Index(['Country', 'concentration', 'concentration_change', 'penetration',
       'representation', 'migration', 'job_posting', 'hiring_rate', 'robots',
       'robot_growth', 'AI_invest_2024', 'AI_total_invest', 'new_AI_startups',
       'new_AI_startups_10yr', 'Talent Score'],
      dtype='object')
```

In [10]:
```
talent.columns = talent.columns.str.replace("_", " ").str.title().str.replace(" ", "_")
talent.columns = [col.replace("Ai", "AI") for col in talent.columns]
```

In [11]:
```
total_raw = total_raw[["Country", "Total_Score"]]
total_softmax = total_softmax[["Country", "Raw_Total", "Softmax_Score"]]
```

In [12]:
```
compute     = compute.add_prefix("Compute_")
industry    = industry.add_prefix("Industry_")
policy      = policy.add_prefix("Policy_")
research    = research.add_prefix("Research_")
talent      = talent.add_prefix("Talent_")
total_raw   = total_raw.add_prefix("TotalRaw_")
total_softmax = total_softmax.add_prefix("TotalSoftmax_")
```

In [13]:
```
compute = compute.rename(columns={"Compute_Country": "Country"})
industry = industry.rename(columns={"Industry_Country": "Country"})
policy = policy.rename(columns={"Policy_Country": "Country"})
research = research.rename(columns={"Research_Country": "Country"})
talent = talent.rename(columns={"Talent_Country": "Country"})
total_raw = total_raw.rename(columns={"TotalRaw_Country": "Country"})
total_softmax = total_softmax.rename(columns={"TotalSoftmax_Country": "Country"})
```

In [14]:
```
replace_map = {
    "Korea": "South Korea",
    "korea": "South Korea",
    "Republic of Korea": "South Korea",
    "republic of korea": "South Korea",
    "South korea": "South Korea",
}
```

```
for df in [compute, industry, policy, research, talent, total_raw, total_softmax]:
    df["Country"] = df["Country"].replace(replace_map)
```

In [15]:
```
combined = compute.copy()

dfs = [industry, policy, research, talent, total_raw, total_softmax]

for df in dfs:
    combined = combined.merge(df, on="Country", how="outer")
```

In [16]:
```
combined.to_csv("Combined_AI_Target_Scores.csv", index=False)
```